Drools

David THIBAU – 2017

david.thibau@gmail.com

Agenda

- Introduction
  - BRMS
  - Les projets KIE
  - Le moteur de règles Drools
  - Installation, Le plugin Eclipse
- Prise en main
  - Session stateless
  - Session stateful
  - Conflits et agenda
- Le langage de règles DRL
  - Éléments principaux
  - Règles et attributs
  - LHS / RHS
  - Requêtes
- Drools Avancé
  - Intégration avec jBPM
  - Drools Fusion et CEP
- Alternatives au DRL
  - DSL
  - Tables de décision
  - Gabarits de règles
- Intégrer Kie
  - Considérations et patterns d'intégration
  - Dépôts de connaissance Maven
  - Knowledge As A Service
  - Drools Workbench
- Annexes

Introduction

BRMS
Les projets KIE
Le moteur de règles Drools
Cas d'utilisation
Installation, Le plug-in Eclipse

Problématique

Aujourd'hui, le principal challenge des applications métier est l'agilité

Les applications doivent pouvoir s'adapter rapidement et réagir :

- A des demandes d'évolutions fonctionnelles
- A des changement de législation
- A des changements d'organisation
- A l'intégration de nouveaux systèmes

Approche traditionnelle

1) Demande de changement

2) Prioritisation et allocation de ressources

3) Analyse

4) Design

5) Code

6) Test/ Debug

7) Déploiement

=> Des changements simples peuvent avoir de longs délais de réalisation

Solutions classiques

Stockage de paramètres de configuration dans des tables (bd ou autres)

=> Ne répond pas à

Si le client habite Paris, est âgé de plus de 55 ans et est client depuis plus de 2 ans, accorder une réduction de 10%

Codage des règles dans l'application

- Maintenance très délicate
- Code spaghetti pouvant être inefficace

Système de gestion de règles

Un **système de gestion de règles (BRMS)** identifie la notion de règle métier comme une ressource pouvant être gérée indépendamment du code applicatif

- Les règles peuvent donc être gérées, versionnées, monitorées
- Elles peuvent être mises à jour par un technicien ou un expert métier sans redéploiement
- Elles peuvent être testées indépendamment
- Elles peuvent être documentées et auditées

Moteur de règle

Les BRMS intègre un **moteur de règle**

C'est un composant qui évaluent des instructions IF-THEN basées sur le modèle métier : les règles

Lorsque les conditions des règles sont satisfaites (*IF*), il exécute les actions associées (*THEN*) qui généralement modifient le modèle

=> La programmation devient alors déclarative

=> La logique métier n'est plus dispersée dans le code mais centralisée dans le repository de règles

Scénario d'utilisation

Service de règle :

– Parses et compile l'ensemble des règles. (Fait en général un seule fois au démarrage de l'application)

Client (code applicatif) :

– Obtient une référence (locale ou service web) sur le service de règle

– Insère des objets (faits) dans la mémoire de travail

– Demande au service de déclencher les règles

– Récupère les objets modifiés par le service de règles

Mise en place

Adopter une solution basée sur les règles nécessite :

- L'identification des règles métier : Expert métier
- Leur organisation et leur implémentation dans un langage de règles : Expert métier / technique
- L'intégration du service de règles à l'application (Build/Déploiement) : Technique
- La mise à disposition d'une interface de gestion des règles (Outil BRMS) : Métier / Technique
- La mise en place de procédure de mise à jour (Automatisation des tests/Déploiement) : Technique

Considérations

- Un moteur de règles s'appuie sur des technologies complexes
- Il est difficile de se baser sur une boite noire
- Comment les règles se déclenchent n'est pas très intuitif
- L'extraction des règles n'est pas toujours simple

Avantages

L'utilisation d'un moteur de règles apporte certains avantages :

- **Programmation déclarative** : Les moteurs de règles permettent de spécifier « Quoi faire » et non pas « Comment le faire ». Ils sont capables de résoudre des problèmes difficiles et en plus de fournir des explications !
- **Séparation de la logique et des données** : Les données sont dans les objets du domaine, la logique centralisée dans un fichier de règles (différent de l'approche OO qui encapsule attributs et méthodes). La logique n'est plus dispersée
- **Centralisation de la connaissance** : Tout est centralisé dans la base de connaissance, relativement lisible et pouvant servir de documentation
- **Règles compréhensibles** : En définissant des langages spécifiques au domaine ou au métier, les règles sont exprimées en langage quasi-naturel et deviennent accessibles aux experts métier

Quand utiliser un moteur de règles ?

Le problème est trop complexe pour le code classique (problèmes d'optimisation par exemple, système expert)

Le problème n'est pas complexe mais aucune solution robuste s'impose.

La logique change souvent, dans ce cas les règles peuvent être changées rapidement sans trop de risques.

Les experts métiers existent mais ne sont pas techniques. Les règles permettent alors d'exprimer la logique métier dans leur propre termes.

Quand ne pas utiliser de moteur de règles ?

Un moteur de règles est juste une partie d'une application complexe, il n'est pas nécessaire de tout implémenter sous formes de règles

Un bon indicateur est le degré de couplage entre les règles.

- Si le déclenchement d'une règle déclenche **invariablement** une chaînes de règles, alors l'implémentation de cette logique à base de règles n'est sûrement pas adaptée, un arbre décisionnel peut suffire

Domaines et cas d'utilisation

Domaines

- Finance et Assurance
- Réglementation, règles gouvernementales
- E-commerce
- Gestion des risques

Cas d'utilisation

- Autorisation d'accès basée sur plusieurs critères (Rôle, propriété de l'entité, organisation, Localité, …)
- Personnalisation d'application (ex : gestion d'une page d'accueil personnalisé d'un site de e-commerce)
- Diagnostique
- Validation complexe
- Workflow / Orchestration
- Problème de routage, Optimisation de planning, de stockage, ...

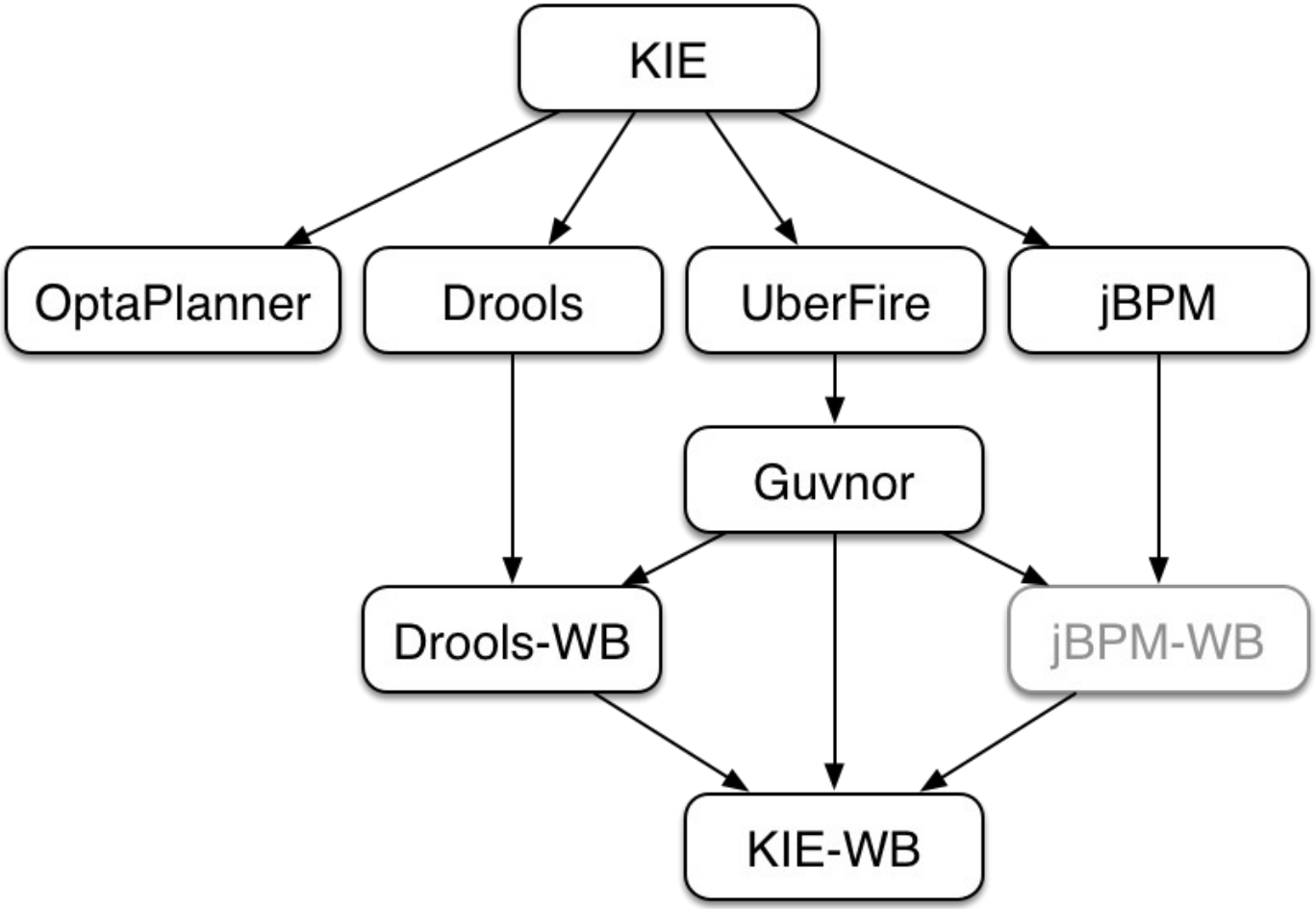
Les projets KIE

Projets KIE

KIE (Knowledge Is Everything) regroupe plusieurs projets qui partagent la même API et les mêmes techniques de construction et de déploiement (basées sur Maven et Git)

- **Drools** : Moteur de règles et traitement complexe d'événements (Complex Event Processing)
- **jBPM** : Moteur de workflow
- **OptaPlanner** (anciennement Drools Planner) Résolution d'heuristique
- **Appformer** : Un framework pour facilement construire des interfaces web : les workbenches (remplacement de Uberfire et de Guvnor)

Projets KIE


KIE
OptaPlanner
Drools
UberFire
jBPM
Guvnor
Drools-WB
jBPM-WB
KIE-WB

Cycle de vie d'un projet KIE

Authoring : Création de la connaissance en utilisant un langage spécifique : DRL, BPMN2, Table de décision, …

Build : Construction d'un artefact de la connaissance déployable (***.jar***)

Test : Test de la connaissance avant déploiement

Déploiement : Déploiement à un emplacement utilisable par les applications clientes (Dépôt similaire à **Maven**)

Utilisation : Le jar est exposée via un objet ***KieContainer***. Pour interagir avec le système, une application créée des ***KieSession*** à partir du conteneur

Exécution : Appels de l'API de *KieSession*

Travaux : Interaction utilisateur avec la *KieSession* via des commandes en ligne ou une interface utilisateur

Exploitation : Surveillance des sessions, monitoring, reporting

API Kie

KieServices : Un singleton donnant accès aux autres services fournis par Kie

KieContainer: Permet le chargement d'un KieModule et de ses dépendances.

KieModule : Définit des base de connaissances (KieBase), des sessions (KieSession) + d'autres objets utilisés par le moteur

KieBase : Contient les ressources KIE comme les règles, les processus, etc.

KieSession : Définit une manière d'interagir avec une base de connaissance

KieServices

```
// Récupérer le singleton KieServices

KieServices kieServices =
  KieServices.Factory.get();

// Instancier un container chargeant les modules

// à partir du classpath

KieContainer container =
  KieServices.getKieClasspathContainer()
```

KieContainer

Différents *KieContainer* sont fournis, ils permettent de charger des modules de différentes façons :

- A partir du classpath
- A partir d’un dépôt maven
- Via une API REST

KieModule

Un *KieModule* configure différentes *KieBase* et *KieSessions*

La configuration est faite

- Via un fichier XML : *META-INF/kmodule.xml*
- Ou programmatiquement

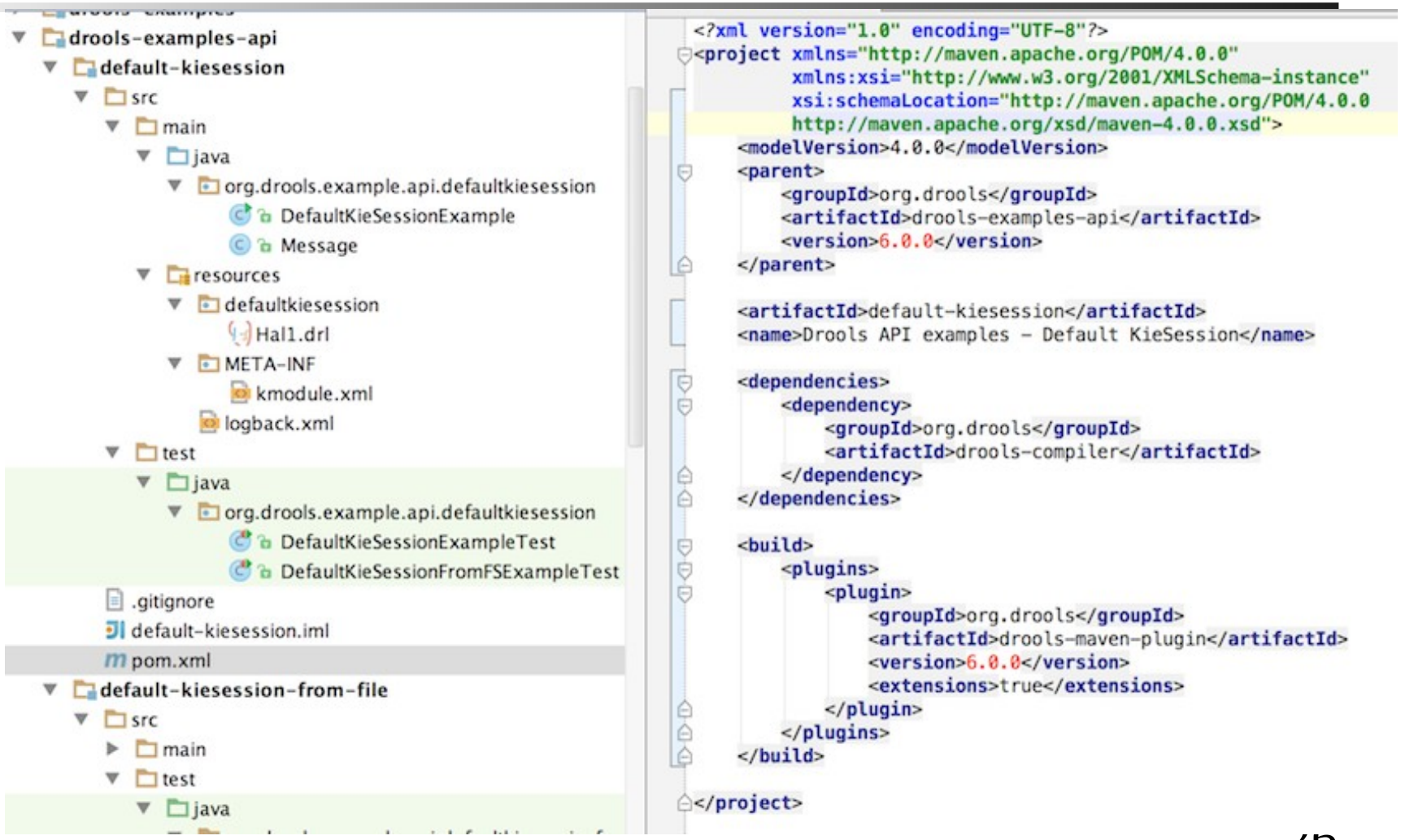
Projet Maven

Typiquement, un projet KIE est un projet Maven avec :

- un *pom.xml* contenant les dépendances applicatives et le *KieRuntime*
- Un fichier *META-INF/kmodule.xml* qui
  - Configure les bases de connaissance en spécifiant les ressources métier (fichiers décrivant les règles ou les processus)
  - Pour chaque base de connaissance, configure une ou plusieurs types de sessions pouvant être créées.

Un plugin Maven permettant de valider les fichier ressources est disponible.

Structure projet



Approche « convention »

Les versions 6+ s'appuient sur une approche de **configuration par convention** plutôt qu'une configuration programmatique

- La configuration la plus simple est alors un fichier *kmodule.xml* vide

=> Lorsque KIE démarre, le classpath container est utilisé :

- tous les fichiers ressources trouvés dans le classpath sont ajoutés à la même base de connaissance.
- 2 types de sessions (stateless et stateful) sont associées à l'unique base de connaissances

Packaging

L’artefact du projet est donc un JAR ou un KJAR qui contient :

- Le descripteur de module *META-INF/kmodule.xml*
- Les fichiers ressources contenant les règles métiers
- Les classes du modèle du domaine.
- Les classes de test permettant de tester les règles

Le code client et le runtime drools permettant d’exécuter les règles est généralement packagé dans un autre projet

Le moteur de règles Drools

Système de production de règle

Un système de production de règle est composé d'une **base de connaissance,** d'un **moteur d'inférence** et d'une **mémoire de travail**

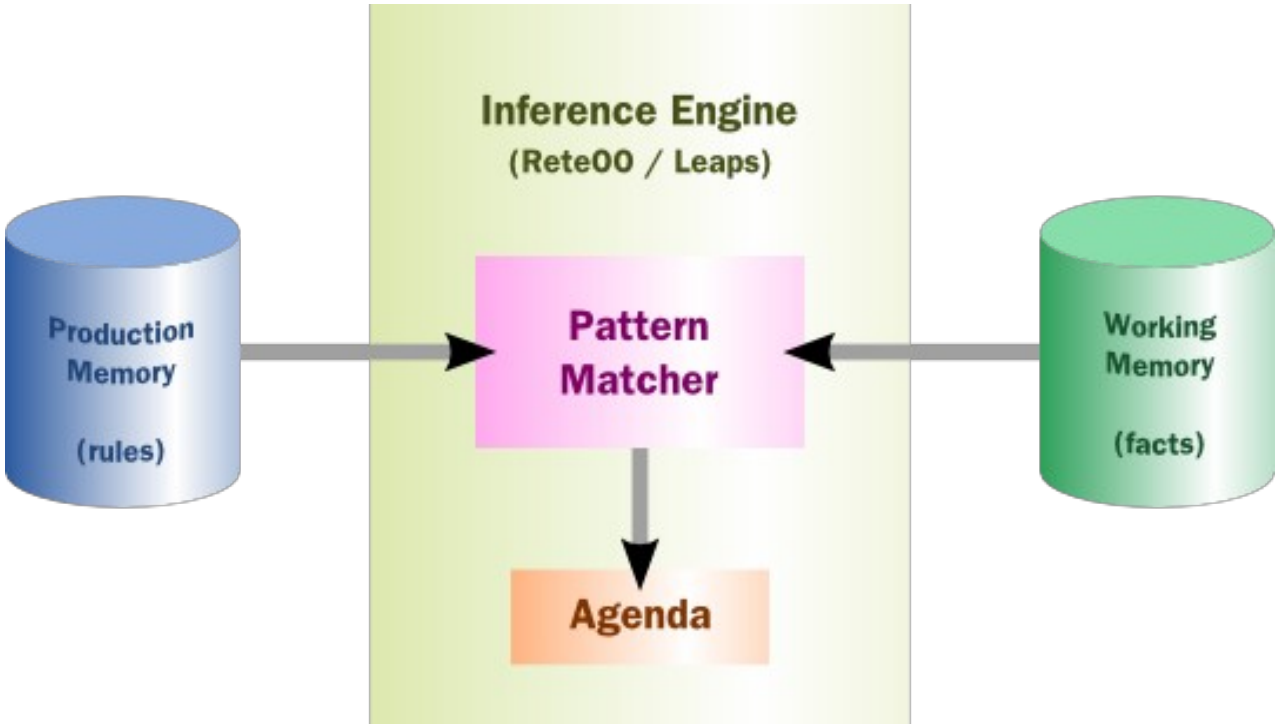
- La base de connaissance regroupe les **règles compilées**
- L'application insère des **faits** (objets du modèle métier) dans la mémoire de travail
- Le moteur d'inférence, capable de manipuler un volume important de règles et de faits, a pour rôle de **comparer les faits aux conditions des règles**, si les conditions des règles sont satisfaites les actions correspondantes sont effectuées.
  Les actions modifient les faits de la mémoire de travail, ce qui peut déclencher l'activation d'autres règles.

Composant Agenda

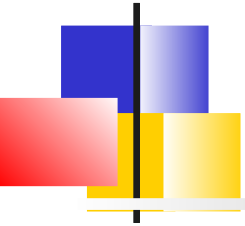
Lors du déclenchement des règles, il se peut que plusieurs règles soient actives simultanément, on dit alors qu'elles sont en conflit.

Le composant **Agenda** est responsable de gérer l'ordre d'exécution des règles en conflit en utilisant une stratégie de résolution de conflit. (priorité ou autre)

Système de production de règles



Algorithmes Drools



Le traitement de comparer les faits aux règles est appelé le **Pattern Matching**, il existe de nombreux algorithmes de pattern matching : *Linear*, *Rete*, *Treat*, *Leaps*.

Drools a implémenté l'algorithme de Rete dans une technologie objet

- Jusqu'à la version 5, Drools utilisait ***ReteOO*** qui se déclenchait à chaque insertion de fait
- A partir de la version 6, Drools utilise ***PHREAK*** par défaut qui doit être explicitement déclenché via l'API (Par ex : `ksession.fireAllRules()`)

Installation

Installation IDE

- Drools fournit des plugins Eclipse qui permet de récupérer toutes les dépendances nécessaires
  - Java 1.5+ est alors requis
  - Eclipse 3.4+
  - Eclipse GEF 3.4

Jars et dépendances

knowledge-api.jar : Interfaces et usines. Identifie clairement l'API utilisateur de l'API moteur. Requis au runtime

knowledge-internal-api.jar : Interfaces et usines à usage interne

drools-core.jar : Le moteur implémentant les algorithmes de *pattern matching*. Requis au runtime

drools-compiler.jar : Compilateur des règles. Requis au runtime si les règles n'ont pas été pré-compilées

drools-jsr94.jar : Couche d'implémentation de la JSR-94 au dessus de drools-compiler. Cependant, en général il est plus facile d'utiliser l'API Drools

drools-decisiontables.jar : Compilateur pour les tables de décision supportant les formats excel et CSV.

Maven, Gradle, Ivy, Buildr, Ant

Les jars sont également disponibles dans les repository Maven et Jboss.

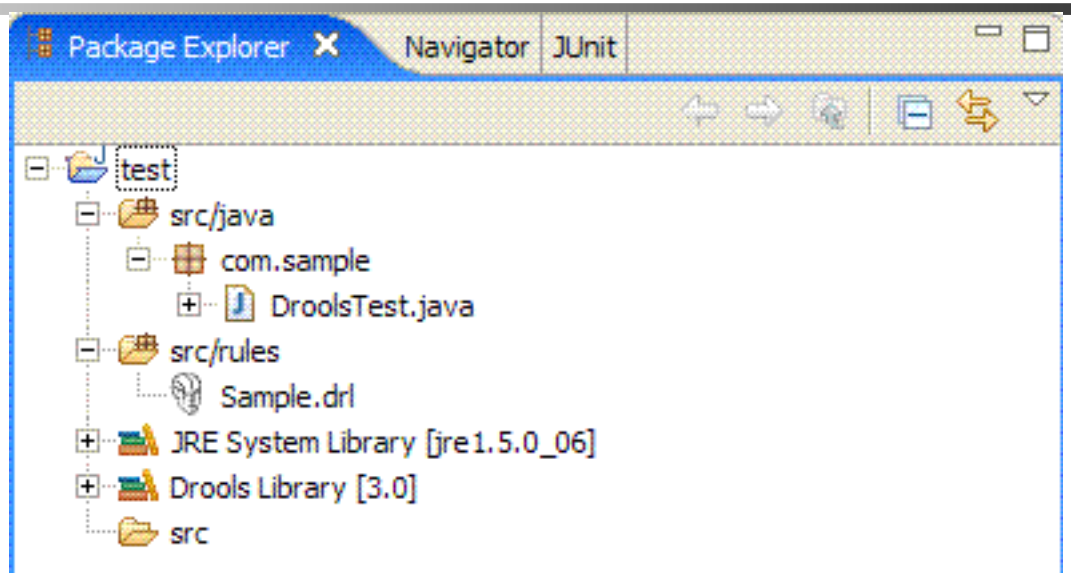
```
<dependencyManagement><dependencies><dependency>
      <groupId>org.drools</groupId>
      <artifactId>drools-bom</artifactId>
      <type>pom</type>
      <version>...</version>
      <scope>import</scope>
    </dependency>
...
  </dependencies></dependencyManagement>
<dependencies><dependency>
  <groupId>org.kie</groupId>
  <artifactId>kie-api</artifactId>
  </dependency>
  <dependency>
  <groupId>org.drools</groupId>
  <artifactId>drools-compiler</artifactId>
  <scope>runtime</scope>
  </dependency>
...
<dependencies>
```

Le plug-in Eclipse

Fonctionnalités

- Éditeur de règle textuel ou graphique, supportant DRL et DSL (coloration, complétion)
- Éditeur graphique pour les processus.
- Assistants pour la création de projet Drools, de fichier *.drl, .brl, .dsl*, de table décisionnelle, de processus
- Éditeur spécifique pour les DSLs gérant les associations DSL/DRL
- Validation de règle : Les règles sont construites à la volée

Projet « rule »



Le plugin ajoute un "Drools Builder" qui permet de valider et compiler les règles lors de tout changement. Si le projet contient beaucoup de règles, il faut peut-être désactiver cette fonctionnalité

Assistants

New
Select a wizard
Wizards:
type filter text
Plug-in Project
General
CVS
Java
JBoss Rules
New Domain Specific Language
New Rule Project
New Rule resource
Plug-in Development
< Back
Next >
Finish
Cancel

Éditeurs de règles

- Éditeur texte
  - Gère les fichiers *.drl* ou *.rule*
  - Support du *dsl*
  - Complétion : *Control+Space*
- Éditeur GUI
  - Gère les fichiers *.brl*

Vue Drools

Pour le debug, le plugin propose plusieurs vues permettant d'inspecter le moteur de règles. Ces vues sont disponibles lors de l'arrêt de l'exécution sur un point d'arrêt (par exemple sur *ksession.fireAllRules()*)

1. La ***Working Memory View*** permet d'inspecter tous les faits insérés
2. L' ***Agenda View*** permet de voir toutes les règles activées dans l'agenda. Pour chaque règle, le nom de la règle et ses variables sont montrées.
3. La ***Global Data View*** permet de voir toutes les globales définies dans la mémoire de travail.

La vue d'audit

La vue audit permet de visualiser un fichier de trace qui a pu être généré grâce au code suivant :

```
KieRuntimeLogger logger =

KieServices.Factory.get().getLoggers().newFileLogger(k
  session, "logdir/mylogfile");

ksession.insert(...);

ksession.fireAllRules();

// stop logging

logger.close();
```

Ou qui a pu être configuré via *kmodule.xml*

Événements

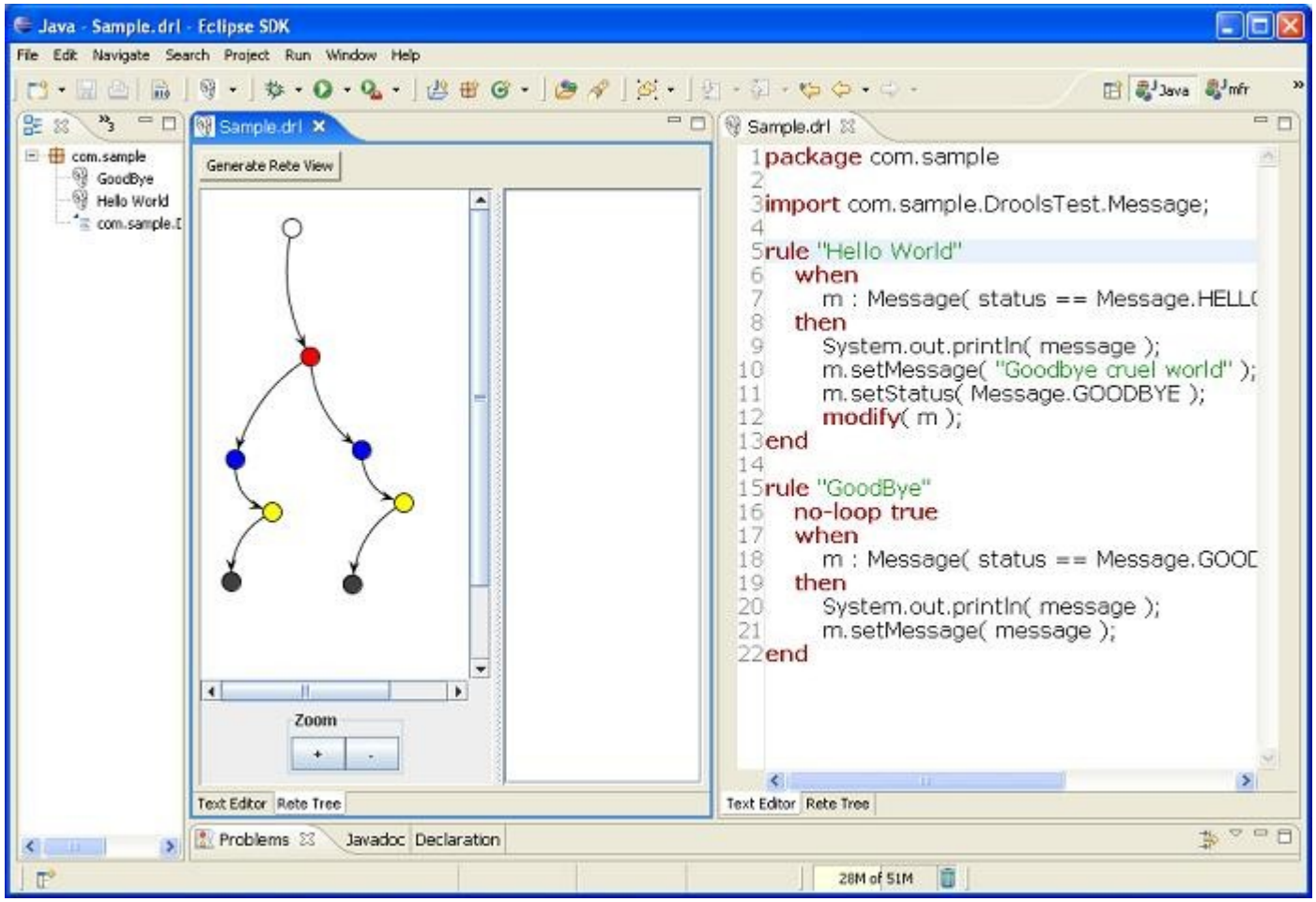
1. Objet inséré :
2. Objet mis à jour:
3. Objet détruit :
4. Activation créé :
5. Activation annulé :
6. Activation exécuté :
7. Séquence de règle démarre ou se termine :
8. Activation ou désactivation d'un groupe de séquence de règle:
9. Ajout ou suppression de packages:
10. Ajout ou suppression de règle :

Exemple

- Activation executed: Rule assignFirstSeat context=[fid:10:10]; count=[fid:11:11]; guest=[fid:8:8]
  - Object asserted (12): [Seating id=1 , pid=0 , pathDone=true , leftSeat=1, leftGuestName=n5, rightSeat=1, rightGuestName=n5]
  - Object asserted (13): [Path id=1, seat=1, guest=n5]
  - Object modified (11): [Count value=2]
    - Activation created: Rule assignFirstSeat context=[fid:10:10]; count=[fid:11:15]; guest=[fid:0:0]
    - Activation created: Rule assignFirstSeat context=[fid:10:10]; count=[fid:11:15]; guest=[fid:1:1]
    - Activation created: Rule assignFirstSeat context=[fid:10:10]; count=[fid:11:15]; guest=[fid:2:2]
    - Activation created: Rule assignFirstSeat context=[fid:10:10]; count=[fid:11:15]; guest=[fid:3:3]
    - Activation created: Rule assignFirstSeat context=[fid:10:10]; count=[fid:11:15]; guest=[fid:4:4]
    - Activation created: Rule assignFirstSeat context=[fid:10:10]; count=[fid:11:15]; guest=[fid:5:5]
    - Activation created: Rule assignFirstSeat context=[fid:10:10]; count=[fid:11:15]; guest=[fid:6:6]
    - Activation created: Rule assignFirstSeat context=[fid:10:10]; count=[fid:11:15]; guest=[fid:7:7]
    - Activation created: Rule assignFirstSeat context=[fid:10:10]; count=[fid:11:15]; guest=[fid:8:8]
  - Object modified (10): [Context state=ASSIGN_SEATS]
    - Activation cancelle[illegible]at context=[fid:10:16]; count=[fid:11:15]; guest=[fid:2:2]
    - Activation cancelle[illegible]at context=[fid:10:16]; count=[fid:11:15]; guest=[fid:4:4]
    - Activation cancelled: Rule assignFirstSeat context=[fid:10:16]; count=[fid:11:15]; guest=[fid:8:8]
    - Activation cancelled: Rule assignFirstSeat context=[fid:10:16]; count=[fid:11:15]; guest=[fid:6:6]
    - Activation cancelled: Rule assignFirstSeat context=[fid:10:16]; count=[fid:11:15]; guest=[fid:1:1]
    - Activation cancelled: Rule assignFirstSeat context=[fid:10:16]; count=[fid:11:15]; guest=[fid:3:3]
    - Activation cancelled: Rule assignFirstSeat context=[fid:10:16]; count=[fid:11:15]; guest=[fid:7:7]
    - Activation cancelled: Rule assignFirstSeat context=[fid:10:16]; count=[fid:11:15]; guest=[fid:5:5]
    - Activation cancelled: Rule assignFirstSeat context=[fid:10:16]; count=[fid:11:15]; guest=[fid:0:0]
    - Activation created: Rule findSeating leftGuestName=[fid:0:0]; rightGuestSex=[fid:8:8]; seatingId=[fid:12:12]; seatingRightGuestName=[fid:12:12]; context=[fid:10:16];
    - Activation created: Rule findSeating leftGuestName=[fid:4:4]; rightGuestSex=[fid:8:8]; seatingId=[fid:12:12]; seatingRightGuestName=[fid:12:12]; context=[fid:10:16];

Show Cause

Vue Rete


Java - Sample.drl - Eclipse SDK
File Edit Navigate Search Project Run Window Help
com.sample
GoodBye
Hello World
Sample.drl
Generate Rete View
Zoom
Text Editor
Rete Tree
Problems
Javadoc
Declaration
28M of 51M
package com.sample
import com.sample.DroolsTest.Message;
rule "Hello World"
when
m : Message(status == Message.HELL
then
System.out.println(message);
m.setMessage("Goodbye cruel world");
m.setStatus(Message.GOODBYE);
modify(m);
end
rule "GoodBye"
no-loop true
when
m : Message(status == Message.GOOD
then
System.out.println(message);
m.setMessage(message);
end

Légende Vue Rete

- Vert : Point d'entrée
- Rouge : ObjectTypeNode
- Bleu : AlphaNode
- Jaune : Adaptateur de l'entrée gauche
- Vert : BetaNode
- Black : Nœud règle

Sélectionner un nœud et visualiser la fenêtre propriétés

Prise en main

Session stateless
Session stateful
Compléments
Le plugin Eclipse

Classes principales

KieServices : Singleton donnant accès aux services *Kie* (*Conteneur, Loggers, stockage, Sérialiseurs, ...*)

KieContainer : Le conteneur responsable de récupérer la configuration et donc le ou la base de connaissance, les sessions, ...

KieModule : La configuration correspondant à un *kmodule.xml*

KieBase : La base de connaissance compilée

KieSession : L'interface de la mémoire de travail.

Types de sessions

Deux types de session Drools sont possibles :

- Session ***stateless*** : *StatelessKieSession. (N'utilise pas l'inférence)*
- Session ***stateful*** : *KieSession*

Session stateless

Stateless Knowledge Session

Les sessions ***stateless*** sont le cas le plus simple car elles n'utilisent pas le moteur d'inférence.
Elles peuvent être assimilées à une fonction à laquelle on passerait des arguments et qui provoquerait un résultat.

Les cas d'utilisation des sessions *stateless* sont généralement :

- La validation : Une personne est elle éligible pour un prêt ?
- Le calcul : Calculer une réduction
- Le routage ou le filtrage : Filtrer des emails, transférer des messages vers des destinations
- ...

Exemple (Modèle métier/ les faits)

```
public class Applicant {
    private String name;
    private int age;
    // getter and setter methods here
}


public class Application {
    private Date dateApplied;
    private boolean valid;
    // getter and setter methods here
}
```

Exemple - Règles

```
package com.company.license

rule "Is of valid age"
when
    Applicant( age < 18 )
    $a : Application()
then
    $a.setValid( false );
end

rule "Application was made this year"
when
    $a : Application( dateApplied > "01-jan-2014" )
then
    $a.setValid( false );
end
```

Pattern matching (Rete)

Lorsqu'une instance d'un *Applicant* est insérée dans le moteur, ce fait est évalué en regard des contraintes des règles.
Dans ce cas, les 2 contraintes de la première règle (contrainte sur le type et sur le champ âge).

Une contrainte sur un type d'objet et une ou plusieurs contraintes sur ses champs est appelé un **pattern**.

Lorsqu'une instance insérée satisfait le pattern, la **conséquence** de la règle s'exécute.

La notation ***$a*** représente une **variable** permettant de référencer l'objet matché dans la conséquence. Ses propriétés peuvent être mises à jour dans la partie conséquence.
Le caractère ('$') est optionnel mais rend l'expression de la règle plus lisible.

Configuration

Il faut fournir un fichier *kmodule.xml* qui est placé dans le répertoire projet ; ***ressources/META-INF/kmodule.xml***

Si on utilise la configuration par défaut, le fichier peut être vide :

```
<?xml version="1.0" encoding="UTF-8"?>

<kmodule xmlns="http://jboss.org/kie/6.0.0/kmodule"/>
```

Création du conteneur

```
// Singleton des services Kie

KieServices kieServices = KieServices.Factory.get();

// Charger le module à partir du classpath

// Compile les fichiers drl trouvés

// Positionne le résultat compilé

KieContainer kContainer =
  kieServices.getKieClasspathContainer();
```

Exécution

```
// Création d'une session stateless
StatelessKieSession ksession = kContainer.newStatelessKieSession();
Applicant applicant = new Applicant( "Mr John Smith", 16 );
Application application = new Application();
assertTrue( application.isValid() );
// Exécution des règles pour ces 2 faits
ksession.execute(
  Arrays.asList( new Object[] { application, applicant } ) );
assertFalse( application.isValid() );
```

BatchExecutor et *CommandFactory*

Les 2 méthodes *execute(Object object)* et *execute(Iterable objects)* sont en fait des raccourcis pour la méthode ***execute(Command command)*** de l'interface *BatchExecutor*

Par exemple, la seconde méthode est équivalent à :

```
ksession.execute(
  CommandFactory.newInsertIterable( new Object[] {
  application, applicant } ) ) ;
```

BatchExecutor et *CommandFactory*

BatchExecutor et *CommandFactory* sont particulièrement utiles pour travailler avec plusieurs commandes et plusieurs identifiants de résultats :

```
List<Command> cmds = new ArrayList<Command>();

cmds.add(CommandFactory.newInsert( new Person( "Mr John Smith" ), "mrSmith" );

cmds.add(CommandFactory.newInsert( new Person( "Mr John Doe" ), "mrDoe" );

BatchExecutionResults results =
   ksession.execute( CommandFactory.newBatchExecution( cmds ) );

assertEquals( new Person( "Mr John Smith" ), results.getValue( "mrSmith" ) );
```

Session stateful

Stateful Session

Les sessions ***Stateful*** ont une durée de vie plus longue et permettent des changements itératifs des faits.

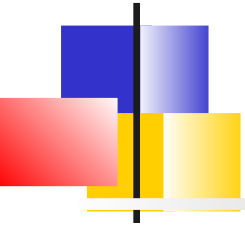
Les cas d'utilisations classiques sont :

- Monitoring : Surveillance des marchés boursiers et achats semi-automatique
- Diagnostique : Découverte de faute, diagnostic médical
- Logistique : Suivi de colis, approvisionnement
- Conformité : Validation d'une législation

Stateful versus stateless

- Contrairement à une Stateless Session, la méthode ***dispose()*** doit être appelée pour éviter les fuites mémoire (*KieBase* contient des références vers les Stateful Sessions)
- Une *Stateful Knowledge Session* est tout simplement appelée ***KieSession***.
- *KieSession* supporte également l'interface *BatchExecutor*
  - Mais la commande ***FireAllRules*** n'est pas automatiquement déclenchée à la fin d'un session stateful.

Exemple – Modèle métier



```
public class Room {
    private String name
    // getter and setter methods here
}
public class Sprinkler {
    private Room room;
    private boolean on;
    // getter and setter methods here
}
public class Fire {
    private Room room;
    // getter and setter methods here
}
public class Alarm { }
```

Exemple – Règle

```
rule "When there is a fire turn on the sprinkler"
when
    Fire($room : room)
    $sprinkler : Sprinkler( room == $room, on ==
  false )
then
    modify( $sprinkler ) { setOn( true ) };
    System.out.println( "Turn on the sprinkler for room
  " + $room.getName() );
end
```

Inférence et *modify*

A la différence de l'exemple sur le *StatelessSession* qui utilisait la syntaxe Java standard pour modifier l'attribut d'un fait, l'utilisation de l'instruction ***modify*** permet d'avertir le moteur des changements effectués sur les faits et donc lui permettre d'effectuer d'autres raisonnements.

C'est ce qu'on appelle **l'inférence**

Opérateur *not*

L'opérateur ***not*** permet de matcher lorsqu'aucune instance de l'objet n'existe dans la base de connaissance :

```
rule "When the fire is gone turn off the sprinkler"
when
    $room : Room( )
    $sprinkler : Sprinkler( room == $room, on == true )
    not Fire( room == $room )
then
    modify( $sprinkler ) { setOn( false ) };
    System.out.println( "Turn off the sprinkler for room " +
  $room.getName() );
end
```

Opérateur *exists*

Son complément ***exist*** teste l'existence d'un objet :

```
rule "Raise the alarm when we have one or more
  fires"
when
    exists Fire()
then
    insert( new Alarm() );
    System.out.println( "Raise the alarm" );
end
```

Instruction *retract*

L'instruction ***retract*** permet de retirer un fait de la base de connaissance

```
rule "Cancel the alarm when all the fires have gone"
  when
    not Fire()
    $alarm : Alarm()
  then
    retract( $alarm );
    System.out.println( "Cancel the alarm" );
end
```

Inférence et maintenance

L'inférence est définie comme l'insertion d'un nouveau fait à partir d'une connaissance précédente :

```
rule "Infer Adult"
when
$p : Person( age >= 18 )
then
insert( new IsAdult( $p ) )
end
```

L'inférence permet de découpler les responsabilités de connaissance et d'encapsuler la connaissance. Cela donne des systèmes beaucoup plus maintenables.

- Par exemple, une autre règle pourra s'appuyer sur le fait d'être adulte plutôt que sur la valeur 18. Le changement de l'âge sera alors facilité

logicalInsert

L'utilisation de ***logicalInsert*** lors d'une règle d'inférence permet de rétracter le fait dés que la clause *when* redevient fausse. Par exemple :

```
rule "Infer Child" when

$p : Person( age < 16 )

then

logicalInsert( new IsChild( $p ) )

end
```

Le fait *IsChild($p)* est automatiquement rétractée dés que la personne atteint 16 ans

Création d'une session *stateful*

```
// Compile les fichiers drl trouvés dans le classpath
// Positionne le résultat compilé dans le conteneur
KieServices kieServices = KieServices.Factory.get();
KieContainer kContainer = kieServices.getKieClasspathContainer();
// Création de la session stateful
KieSession ksession = kContainer.newKieSession();
```

Exécution

```
String[] names = new String[]{"kitchen", "bedroom", "office",
   "livingroom"};
Map<String,Room> name2room = new HashMap<String,Room>();
for( String name: names ){
    Room room = new Room( name );
    name2room.put( name, room );
    ksession.insert( room );
    Sprinkler sprinkler = new Sprinkler( room );
    ksession.insert( sprinkler );
}

ksession.fireAllRules() ;

> Everything is ok
```

FactHandle

Un ***FactHandle*** permet d'obtenir une référence sur un fait inséré.

```
Fire kitchenFire = new Fire( name2room.get( "kitchen" ) );
Fire officeFire = new Fire( name2room.get( "office" ) );
FactHandle kitchenFireHandle = ksession.insert( kitchenFire );
FactHandle officeFireHandle = ksession.insert( officeFire );
ksession.fireAllRules() ;
```

```
> Raise the alarm
> Turn on the sprinkler for room kitchen
> Turn on the sprinkler for room office
```

Cette référence permettra de retirer le fait plus tard :

```
ksession.retract( kitchenFireHandle );
ksession.retract( officeFireHandle );
ksession.fireAllRules() ;
```

```
> Turn on the sprinkler for room office
> Turn on the sprinkler for room kitchen
> Cancel the alarm
> Everything is ok
```

Conflits et agenda

Méthodes versus Règles

Méthodes :

- Elles sont appelées explicitement
- Des instances spécifiques sont passées en argument
- Un appel provoque une exécution unique

Règles :

- Elles s'exécutent dés lors que des données correspondantes sont insérées dans le moteur (Drools5)
- Elles ne sont jamais appelées explicitement
- Des instances spécifiques ne peuvent pas être passées en argument
- Un règle peut se déclencher une fois, plusieurs fois ou aucune fois.

Activations, Agenda et Conflits **(Drools5)**

Les faits sont évalués dés leurs insertions, cependant les règles ne sont déclenchées qu'au moment de l'appel à *fireAllRules()*.

Entre temps, les règles et les faits matchés sont placés dans l'agenda pour une exécution ultérieure.

Une entrée de l'agenda est appelée une **Activation**

L'agenda est donc une table des activations susceptibles d'être déclenchées lors de l'appel à *fireAllRules()*

L'ordre des activations est par défaut arbitraire, ainsi lorsqu'il y a plusieurs activations dans l'agenda, celles-ci sont dites en **conflit** et une **stratégie de résolution** de conflit doit être appliquée pour déterminer l'ordre des activations.

Activations, Agenda et Conflits **(Drools6)**

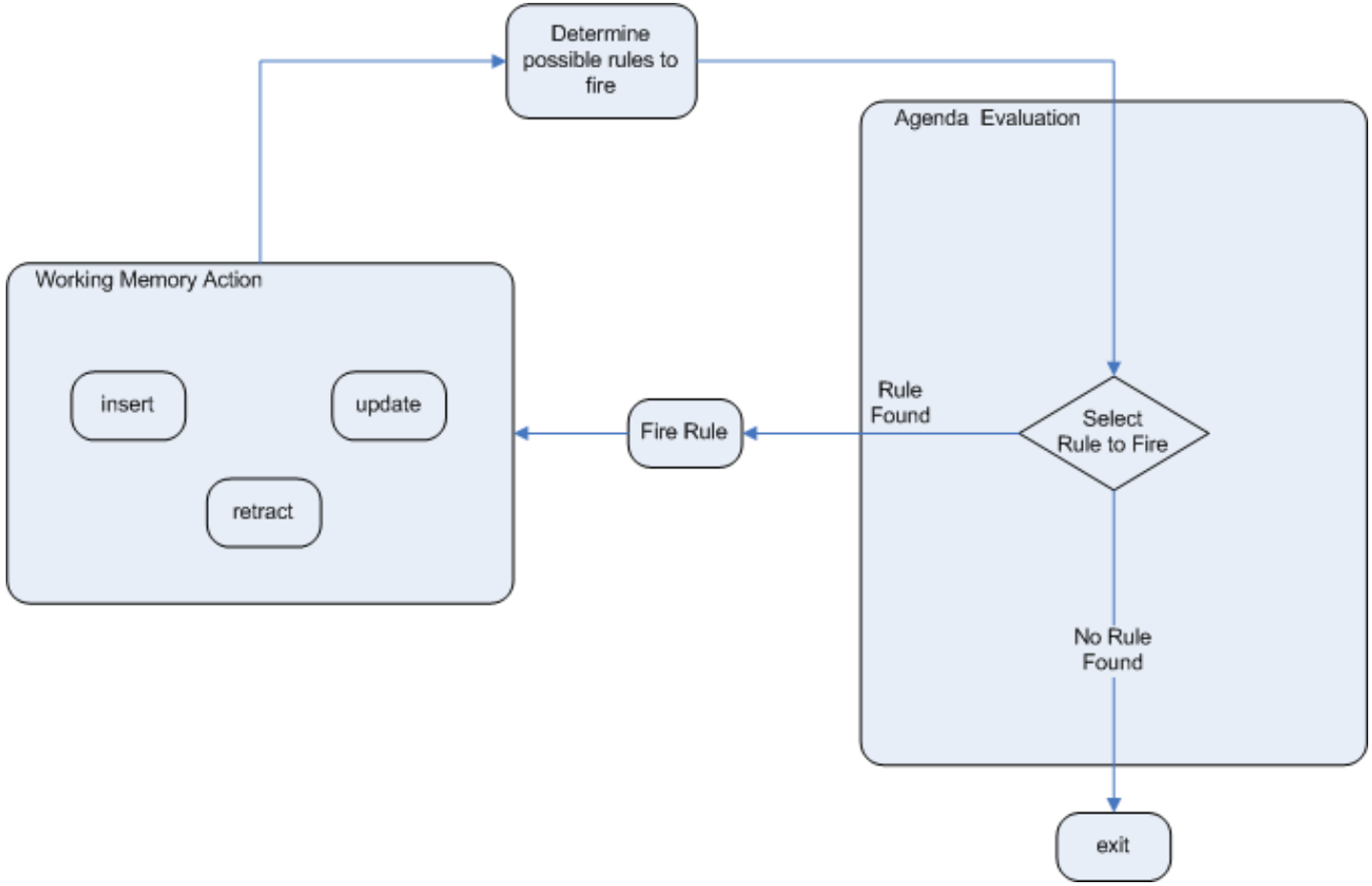
Lors de l'appel à *fireAllRules()*, les règles sont évaluées indépendamment les unes des autres

L'agenda choisit, parmi les règles activées, une seule règle en fonction du **groupe d'activation actif** et des attributs ***salience*** des règles.
Si 2 règles ont la même propriété , c'est l'ordre de déclaration qui joue

La règle est ensuite déclenchée.

Si la conséquence de la règle modifie un fait de la mémoire de travail, l'inférence se produit

Cycle d'évaluation de l'Agenda



Les groupes d'agenda

Les **groupes d'agenda** permettent de partitionner les règles dans des groupes.

Les groupes sont gérés sous forme de pile

Au départ, la pile contient le groupe main

Lorsqu’un groupe est activé, il est placé au dessus de la pile

Lors d’un déclenchement de règles, l'agenda évalue les règles du groupe placées sur le haut de la pile

Lorsque plus aucune règle de groupe n’est activée, l'agenda dépile le groupe courant.

Listeners, channels, entry-points

Event Model

Kie fournit un modèle d'événement qui permet à des listeners de réagir à des événements moteurs tels que le déclenchement d'une règle, l'insertion d'un fait, etc.

Cela permet de séparer les activités de trace ou d'audit, de l’expression des règles

L'interface *KieRuntimeEventManager* est implémentée par *KieRuntime* qui fournit 2 interfaces:

- ***RuleRuntimeEventManager***
- et ***ProcessEventManager***.

listeners fournis par Drools

Drools fournit 2 listeners :

- *DebugRuleRuntimeEventListener* et *DebugAgendaEventListener* qui affiche des messages de debug

KieRuntimeLogger utilise également le système évenementiel pour générer un fichier de traces visible par la vue Audit d'Eclipse

```
KieRuntimeLogger logger =
KieServices.Factory.get().getLoggers().newFileLogger(ksession,
  "logdir/mylogfile");
...
logger.close();
```

Exemple de Listener personnalisé

```
ksession.addEventListener( new
  DefaultAgendaEventListener() {
public void afterMatchFired(AfterMatchFiredEvent event) {
super.afterMatchFired( event );
System.out.println( event );
}
});


ksession.addEventListener( new
  DebugRuleRuntimeEventListener() );
```

Configuration via *kmodule.xml*

```
<kmodule xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xmlns="http://jboss.org/kie/6.0.0/kmodule">
  <kbase name="KBase1" default="true" eventProcessingMode="cloud" equalsBehavior="equality">
    <ksession name="KSession2_1" type="stateful" default="true/">
    <ksession name="KSession2_1" type="stateless" default="false" beliefSystem="jtms"/>
</kbase>
<kbase name=""KBase2" default="false" eventProcessingMode="stream" equalsBehavior="equality"
 declarativeAgenda="enabled"
packages="org.domain.pkg2,org.domain.pkg3" includes="KBase1">
<ksession name="KSession2_1" type="stateful" default="false" clockType="realtime">
  <fileLogger file="drools.log" threaded="true" interval="10"/>
  <listeners>
    <ruleRuntimeEventListener type="org.domain.RuleRuntimeListener"/>
    <agendaEventListener type="org.domain.FirstAgendaListener"/>
    <agendaEventListener type="org.domain.SecondAgendaListener"/>
  </listeners>
</ksession>
</kbase>
</kmodule>
```

Channels

Un **channel** est un moyen de transmettre des données de la mémoire de travail au monde extérieur.

- Alternative aux variables globales (Voir +loin).

Un *Channel* est l'implémentation d'une interface Java interface définissant une seule méthode :

```
void send(Object object)
```

Les Channels ne peuvent être utilisées que dans la partie conséquence des règles

Registration

Les channels doivent être enregistrés programmatiquement :

```
ksession.registerChannel("audit-channel",
auditChannel);
```

Ensuite, utilisés dans les règles :

```
rule "Send Suspicious Operation to Audit Channel"
when
$so: SuspiciousOperation()
then
channels["audit-channel"].send($so);
end
```

Points d’entrée

Les ***entry points*** sont un moyen de partitionner la mémoire de travail

Les règles peuvent exprimer des conditions sur les événements à partir d'une source particulière

Les points d'entrée sont déclarés implicitement en les utilisant dans les règles

A l'insertion, le point d'entrée peut être spécifié:

```
ksession.getEntryPoint("myEntryPoint").insert(new
  Object());
```

Le langage de règle : DRL

Principaux éléments
Attributs des règles
LHS
RHS
Requête
DSL
Tables de décisions

Principaux éléments

Fichier de règles

Un fichier de règles est un simple fichier texte avec l'extension ***.drl***,

Sa structure est la suivante :

- **package** : Espace de nom
- **imports :** Types Java utilisés
- **declare :** Déclaration interne de nouveaux types
- **globals :** Variables globales accessiies de l'extérieur
- **functions :** Réutilisation de logique
- **queries :** Recherche de faits
- **rules :** Règles

Il est également possible de répartir les règles sur plusieurs fichiers qui ont alors généralement l'extension *.rule*

Structure d’une règle

```
rule "name"
    attributs
    when
        LHS
    then
        RHS
end
```

La ponctuation ", les retours à la ligne sont optionnels

Les attributs sont optionnels

LHS est la partie conditionnelle de la règle

RHS est un bloc qui permet de spécifier en différents dialectes un code à exécuter.

Mots-clés

Mots-clés :

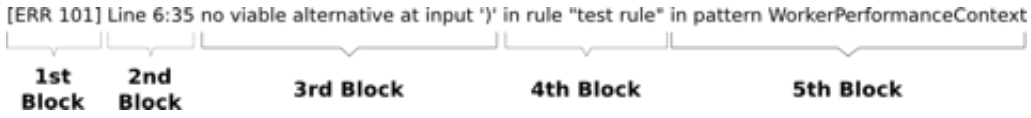
- Hard (*true, false, accumulate, collect, from, null, over, then, when*). Ne peuvent pas être utilisés comme identifiant d'objets
- Soft : seulement reconnu dans leur contexte (*package, import, attributes, rule, ...)*

Le caractère **d'échappement** est ` :
Holiday(`when` == "july")

Commentaires :

- ligne : **#** ou **//**
- multi-lignes : **/* .... */**

Message d'erreur



1er bloc : **Code d'erreur**

2nd bloc : Information de **colonne et ligne**.

3ème bloc : **Description** de l'erreur.

4ème bloc optionnel : Premier **contexte** de l'erreur. Généralement, la règle, la fonction, la requête ou l'erreur s'est produite.

5ème bloc optionnel : Identifie le **pattern** où l'erreur s'est produite.

Package

Un ***package*** regroupe un ensemble de règles, d'imports et de globales qui sont en relation.

Un *package* représente un espace de nom (le nom du package) et doit être unique dans la base de connaissance. Il suit les conventions de nommage des *packages* Java

- Il est possible de répartir les règles d'un même package dans plusieurs fichiers. Dans ce cas, un seul fichier contient la configuration du package.
- Les éléments déclarés dans un *package* peuvent être dans n'importe quel ordre, à l'exception de l'instruction *package* .
- Les ';' sont optionnels.

Import

Les instructions **import** sont similaires aux *imports* dans Java.

Il est nécessaire de spécifier le nom complet des types d'objets que l'on utilise dans les règles.

Drools importe automatiquement les classes Java du package du même nom ainsi que le package *java.lang*.

Global

Avec le mots-clé **global**, on définit des variables globales.

- Elles peuvent être utilisées dans les conséquences des règles.
- Les variables globales ne sont pas insérées dans la mémoire de travail et ne doivent donc pas être utilisées comme conditions dans les règles sauf en tant que constante
- Le moteur n'est pas prévenu lors d'un changement de valeur d'une variable globale

Exemple

```
global java.util.List myGlobalList;

rule "Using a global"
when
eval( true )
then
myGlobalList.add( "Hello World" );
end

-----

List list = new ArrayList();
WorkingMemory wm = rulebase.newStatefulSession();
wm.setGlobal( "myGlobalList", list );
```

Fonctions

Les **fonctions** permettent d'insérer du code dans les fichiers de règles.

- Elles ne peuvent rien faire de plus que des classes *Helper*. Le seul avantage d'utiliser des fonctions est de centraliser la logique à un seul endroit.
- Les fonctions sont utilisées pour invoquer des actions dans la partie conséquence des règles .

Exemple

```
function String hello(String name) {
  return "Hello "+name+"!";
}

rule "using a static function"
when
eval( true )
then
System.out.println( hello( "Bob" ) );
end
```

Déclarations

2 types de déclarations sont possibles :

- La déclaration de **nouveaux types** : *Drools* fonctionne directement avec des POJOs comme faits. Il est donc possible de définir le modèle métier directement dans les règles ou de créer des objets modèles qui n'ont d'utilité que lors du raisonnement. Drools, au moment de la compilation, génère le bytecode Java qui implémente le nouveau type
- La déclaration de **méta-données** ou **annotations**: Les faits ou leurs attributs peuvent être annotés. Les annotations peuvent être utilisées lors du raisonnement ou lors de la recherche de fait.

Déclaration de nouveaux types

```
declare Address
   number : int
   streetName : String
   city : String
end


declare Person
    name : String
    dateOfBirth : java.util.Date
    address : Address
end
```

Accès aux types déclarés

Il est possible d'accéder aux type déclarés à partir du code applicatif via l'interface ***org.drools.definition.type.FactType***

```
// Récupérer le type de données
FactType personType = kbase.getFactType( "org.drools.examples",
                                         "Person" );
// Instancier
Object bob = personType.newInstance();


// Positionner les attributs
personType.set( bob, "name", "Bob" );
personType.set( bob, "age", 42 );
```

Déclaration de méta-données

Le caractère **@** est utilisé

Les méta-données peuvent concerner le type ou un de ses attributs, un nouveau type ou un type existant.

```
declare Person
    @author( Bob )
    @dateOfCreation( 01-Feb-2009 )

    name : String @key @maxLength( 30 )
    dateOfBirth : Date
    address : Address
end
```

Ou sur un type existant

```
declare Person
    @author( Bob )
    @dateOfCreation( 01-Feb-2009 )
end
```

Utilisation des méta-données

Drools permet la déclaration de méta-donnée **arbitraires**.

Les méta-données peuvent être utilisées par les requêtes.

Certaines méta-données sont prédéfinies et ont un sens pour le moteur. Elles sont surtout utile pour *Drools-fusion* :

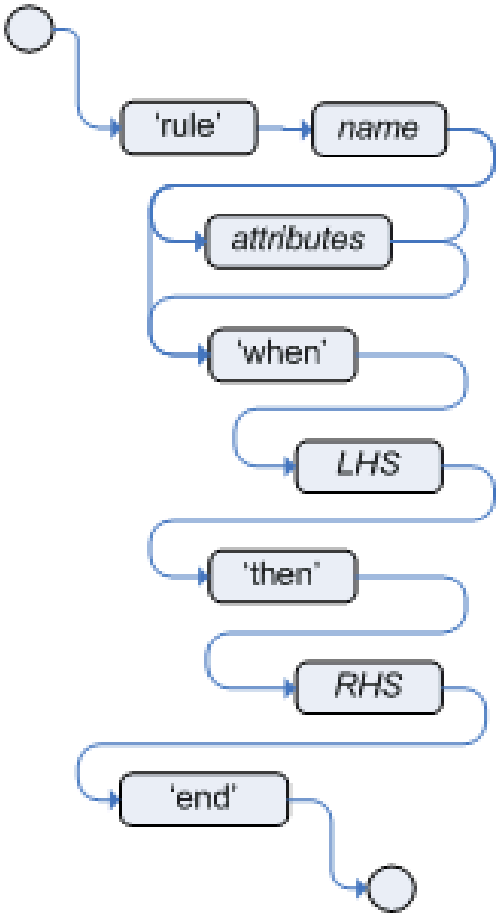
@role, @timestamp, @duration, ...

Exemple d'utilisation des méta-données

```
StatefulKnowledgeSession ksession= createKSession();

ksession.fireAllRules(new AgendaFilter() {

  public boolean accept(Activation activation) {

    Map<String, Object> metaData =
   activation.getRule().getMetaData();

    if (metaData.containsKey("LegalRequirement")) {

      System.out.println(metaData.get("LegalRequirement"));

    }

    return true;

  }

});
```

Règles et attributs

Règle



Syntaxe

- Une règle doit avoir un nom, unique à l'intérieur du *package*.
- Le nom peut contenir des espaces si il est délimité par ".
- La partie gauche de la règle (*Left Hand Side*) ou condition suit le mot-clé ***when***
- La partie droite (*Right Hand Side*) ou conséquence suit le mot-clé ***then***
- La règle se termine par le mot-clé ***end***.
- Les règles ne peuvent pas être imbriquées.

Attributs des règles (1)

no-loop (booléen, false) : Lorsque la conséquence de la règle modifie un fait, il peut provoquer l'activation de la règle une nouvelle fois. La récursion peut donc être évitée avec l'attribut *no-loop* à *true*.

salience (Entier, 0) : Chaque règle a un attribut *salience* de type entier qui détermine la priorité de la règle dans l'agenda.

dialect (String, "java" or "mvel") : Le dialecte est en général spécifié au niveau du package. Cet attribut surcharge la définition au niveau package.

Attributs des règles (2)

agenda-group (String, MAIN) : Cet attribut permet de partitionner l'Agenda et de contrôler l'exécution. Seules les règles du groupe d'agenda qui a acquis le focus sont autorisées à se déclencher.

activation-group (String) : Les règles appartenant au même groupe d'activation sont exclusives. La première règle qui se déclenche annule les autres.

ruleflow-group (String) : Permet de grouper plusieurs règles. Les règles de ce groupe ne seront activées que lorsque le processus sera dans un nœud particulier.

Attributs des règles (3)

auto-focus (booléen, false) : Quand une règle est activée avec l'attribut auto-focus positionné, le groupe indiqué par un de ses attributs (agenda-group ou activation-group) gagne le focus.

lock-on-active (booléen, false) : Lorsqu'un groupe (*ruleflow* ou *agenda*) devient actif, toutes les règles de ce groupe qui ont l'attribut *lock-on-active* positionné ne seront plus activées dans le futur quelque soit l'origine de la mise à jour. Elles pourront être réactivées à la réactivation de leur groupe.

date-effective (String définissant une date) : Une règle ne peut être activée que si la date courante est supérieure à la date effective.

date-expires (String définissant une date) : Une règle ne peut plus être activée que si la date courante est supérieure à la date d'expiration.

Timer

Drools supporte dorénavant les ***timers*** basés sur des intervalles ou exprimés par des expressions *cron*. Les *timers* remplacent l'attribut *duration*.

```
timer ( int: <initial delay> <repeat interval>? )

timer ( cron: <cron expression> )
```

Exemple

```
rule "Send SMS every 15 minutes"
    timer (cron:* 0/15 * * * ?)
when
    $a : Alarm( on == true )
then
    channels[ "sms" ].insert( new Sms( $a.mobileNumber, "The alarm is still on" );
end
```

fireUntilHalt()

Afin que les règles utilisant des *timers* puisse être déclenchées, le moteur doit être actif.

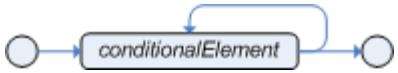
Dans ce cas, il ne faut pas appeler *fireAllRules()* mais ***fireUntilHalt()*** qui évalue les règles tant qu'il ne reçoit pas de signal *halt*

Dans ce cas, arrêter le moteur peut se faire :

- Via une règle : ***drools.halt()***
- Via Java : ***ksession.halt()***
  Dans ce cas, la méthode *fireUntilHalt* est généralement lancé dans une thread indépendante afin que le code Java puisse l'arrêter

LHS

LHS



La partie LHS est la partie conditionnelle de la règle.

Elle est constituée de zéro ou plusieurs éléments conditionnels

- Si aucune condition, la LHS est évaluée à *true* et sera activée dés la création de la mémoire de travail
- Les éléments conditionnels sont constitués de ***patterns*** qui sont implicitement reliés par une logique *and*

Pattern

Un pattern consiste en :

- Un pattern binding permettant de créer une **variable** utilisée dans la règle, le caractère **$** est facultatif mais recommandé
- Une restriction sur le **type** (Un fait, une interface, une classe abstraite)
- Un ensemble de **contraintes** reliées par des opérateurs

Ex : `$c : Cheese()`

Syntaxe des contraintes

2 types de syntaxe sont disponibles :

- Les contraintes de **champ**
  - Portent sur un seul attribut
  - Sont combinés avec les opérateurs && et || et ()
- Les contraintes **groupes**
  - Portent sur plusieurs attributs d'un même fait
  - Sont combinées avec l'opérateur ',' (qui signifie && mais qui a une priorité plus faible lors de l'évaluation)

Contraintes de champ

Une **contrainte de champ** exprime une restriction sur une propriété de l'objet accessible par des getter/setter, il est possible de binder le champ sur une variable

3 types de restriction sont possibles :

- **Valeur unique** : le champ est comparé à une seule valeur
- **Plusieurs valeurs** : le champ est comparé à plusieurs valeurs
- **Multi-contraintes** : Plusieurs contraintes sont spécifiées sur le champ

La valeur du champ peut être de type String, numérique, date (format "*dd-mmm-yyyy*" par défaut), *boolean* ou *Enum* des tests sur *null* peuvent être effectués ainsi que sur des retours de méthode

Contrainte simple valeur

Les opérateurs pouvant être utilisés sont : <, <=, >, >=, ==, *!=, contains, not contains, memberof, not memberof, matches (expr. régulière), not matches*

```
Cheese( quantity == 5 )

Cheese( bestBefore < "27-Oct-2009" )

Cheese( type == "camembert" )

Cheese( from == Enum.COW)

Cheese( type matches "(Buffalo)?\S*Mozarella" )

CheeseCounter( cheeses contains "stilton" )

CheeseCounter( cheese memberOf $matureCheeses )

Person( likes : favouriteCheese ) Cheese( type == likes )

Person( girlAge : age, sex == "F" ) Person( age == ( girlAge + 2), sex ==
   'M' )
```

Contrainte valeurs multiples

Les opérateurs *in* et *not in*, permettent de spécifier plusieurs valeurs séparées par ","

```
Person( $cheese : favouriteCheese )

Cheese( type in ( "stilton", "cheddar", $cheese ))
```

Contraintes multiples

Les contraintes multiples permettent de spécifier plusieurs restrictions sur le champ reliées par les opérateurs '&&' ou '||' et des parenthèses

```
Person( age > 30 && < 40 )
```

```
Person( age ( (> 30 && < 40) ||
              (> 20 && < 25) ) )
```

```
Person( age > 30 && < 40 || location == "london" )
```

Contraintes groupes

La virgule **,** permet de séparer les contraintes de groupes et équivaut à un AND moins prioritaire :

```
Person( age > 50, weight > 80, height > 2 )
```

L'opérateur virgule ne peut être imbriqué dans une expression composite :

```
Person( ( age > 50, weight > 80 ) || height > 2 )
  // => ERREUR de compilation
```

Autres opérateurs

not : Il n'y a aucun fait dans la mémoire de travail correspondant à ces restrictions

exists : Il y a au moins un fait dans la mémoire de travail correspondant à ces restrictions

forall : Tous les faits de la mémoire de travail correspondant à la première restriction satisfont les autres restrictions

from : Permet de comparer des données autres que l'intégralité de la mémoire de travail (Exemple requête)

collect : Permet de raisonner sur une collection d'objets

accumulate : Permet d'effectuer une fonction d'agrégation sur une collection d'objets

Exemples

```
#Il n’y a aucun bus de couleur rouge dans la mémoire
not Bus(color == "red")

#Il y a au moins un bus 42 de couleur rouge dans la mémoire
exists ( Bus(color == "red", number == 42) )

#Tous les bus anglais sont rouge
forall( $bus : Bus( type == 'english')  Bus( this == $bus, color =
   'red' ) )

# Les adresses avec le bon code postal
# qui sont déténues par des personnes de la mémoire
Person( $personAddress : address )
Address( zipcode == "23920W") from $personAddress
```

Exemples

```
# Construction d'une liste de maman
# Toutes les personnes de sexe féminin ayant des enfants
$mothers : LinkedList()
from collect( Person( gender == 'F', children > 0 ) )

# Toutes les commandes dont le total est supérieur à 100
$order : Order()
$total : Number( doubleValue > 100 )
from accumulate( OrderItem( order == $order, $value : value ),
   sum( $value ) )
```

Définition de requête

Une **requête** permet de rechercher des faits présents dans la base de connaissance

Une requête peut être paramétrée.

Les noms des requêtes sont globales à la base de connaissance => Pas de nom identique même dans des packages différents

Sa définition est identique à la partie gauche d'une règle :

```
query "people over the age of x"  (int x)
    person : Person( age > x )
end
```

Utilisation

Le résultat d'une requête est obtenu par :
ksession.getQueryResults("name")

Il est ensuite possible d'itérer sur les lignes retournées

```
QueryResults results = ksession.getQueryResults( "people over the age of x" ,30 );
System.out.println( "we have " + results.size() + " people over the age  of 30" );

System.out.println( "These people are are over 30:" );
for ( QueryResultsRow row : results ) {
    Person person = ( Person ) row.get( "person" );
    System.out.println( person.getName() + "\n" );
}
```

RHS

RHS

La partie droite contient une liste d'actions à exécuter.

En général, pas de code conditionnel car la règle doit être « atomique » (sinon séparer en plusieurs règles)

Les opérations peuvent agir sur la mémoire de travail et donc déclencher l’inférence :

- Ajout de faits
- Suppression de faits
- Modification de faits

Macro-méthodes

Drools propose plusieurs macro-méthodes évitant de récupérer les références des faits que l'on désire mettre à jour :

- ***update(object, handle)*** : Notifie le moteur qu'un objet a été modifié et que les règles doivent être reconsidérée
- ***update(object)*** : idem, mais le test d'égalité est utilisé par le moteur.
- ***insert(new Something())*** : Insère un nouveau fait.
- ***insertLogical(new Something())*** : idem que *insert* mais l'objet est automatiquement supprimé de la mémoire de travail si la règle n'est plus valable.
- ***retract(handle)*** : Retire un fait de la mémoire de travail
- ***retract(object)*** : Idem avec test d'égalité

Instructions *modify*

L'instruction ***modify*** permet d'appeler plusieurs setters sur un objet du modèle

```
modify ( <fact-expression> ) {
    <expression> [ , <expression> ]*
}
```

Exemple :

```
modify( $stilton ){
  setPrice( 20 ),
  setAge( "overripe" )
}
```

drools et KieHelper API

La variable prédéfinie ***drools*** de type *KieHelper* permet d'appeler d'autres méthodes utiles :

- ***drools.getWorkingMemory()*** retourne l'objet *WorkingMemory* ou session.
- ***drools.halt()*** stoppe l'exécution de la règle
- ***drools.setFocus(String s)*** positionne le focus sur un groupe de l'agenda.
- ***drools.getRule().getName()*** retourne le nom de la règle
- ***drools.getTuple()*** retourne le *n-uplet* de faits correspondant à l'exécution de la règle
- ***drools.getActivation()*** retourne l'activation.
- ***insert(Object o)***, ***update(Object o)*** et ***retract(Object o)***

kcontext et KieRuntime API

Toute l'API de la base de connaissance est exposée via la variable prédéfinie ***kcontext*** de type *KieContext*.
Sa méthode ***getKieRuntime()*** retourne un objet de type *KieRuntime* qui propose d'autres méthodes utiles pour la partie RHS :

- ***getAgenda()*** retourne une référence sur l'agenda de la session
- ***getQueryResults(String query)*** retourne le résultat d'une requête
- ***addEventListener, removeEventListener*** : enregistrement de listeners pour la mémoire de travail ou l'agenda.
- ***getKnowledgeBase()*** retournee l'objet *KnowledgeBase* .
- Gestion des variables ***globales*** avec *setGlobal(...)*, *getGlobal(...)* et *getGlobals()*.
- ***getEnvironment()*** : l'environnement d'exécution et ses propriétés

Drools avancé

Intégration avec jBPM
Drools Fusion et CEP

Introduction

Drools et jBPM se complètent, permettant aux utilisateurs finaux de décrire les connaissances métier à l'aide de différents paradigmes: les règles et les processus

Les projets partagent :

- La même API
- Les mêmes patterns d’intégration avec les applications clientes
- Les mêmes mécanismes de construction et de déploiement

Acceder aux processus à partir des règles

Côté action, une règle a accès à *kcontext* et *kcontext.getKieRuntime()*

Avec cette référence d'objet, il est possible de:

- créer, annuler et signaler des processus
- Accédez au *WorkItemManager* pour terminer des *WorkItems*

Exemple

```
rule "Validate OrderLine Item's cost"
  when
    $ol: OrderLine()
  then
    Map<String, Object> params = new HashMap<String, Oject>();
    params.put("requested_amount", $ol.getItem().getCost());
    kcontext.getKieRuntime().startProcess("simple", params);
end
```

Les instances de processus comme faits

- L'insertion d'instances de processus en tant que faits dans le moteur de règles permet d'écrire des règles sur nos processus ou groupes de processus
- Le listener ***RuleAwareProcessEventLister***, fourni par jBPM, insère automatiquement les *ProcessInstances* et les met à jour chaque fois qu'une variable est modifiée
- Les processus doivent inclure des activités asynchrones

Exemple

```
rule "Too many orders for just our Managers"
  when
    List($managersCount:size > 0) from collect(Manager())
    List(size > ($managersCount * 3)) from
      collect(WorkflowProcessInstance(processId == "process-
order"))
  then
    //There are more than 3 Process Order Flows per manager.
    // Please hire more people :)
end
```

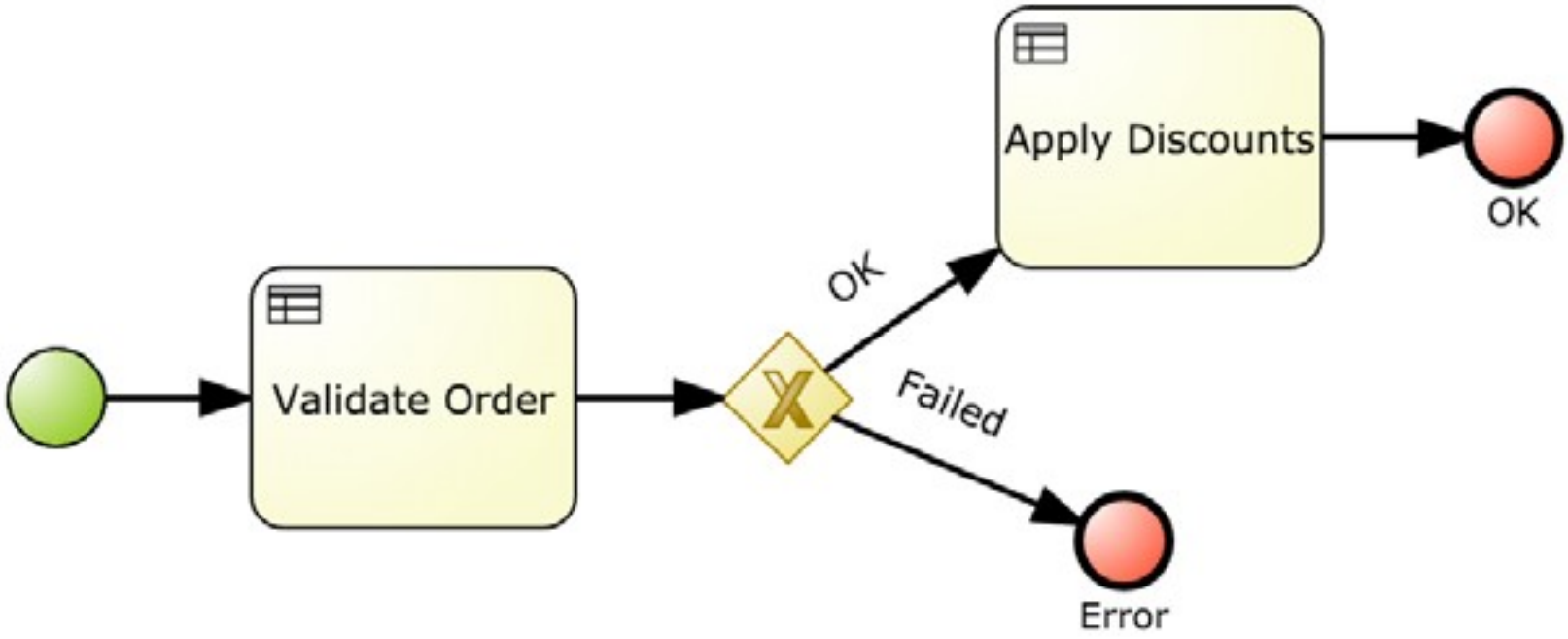
BPMN2 Business Rule Tasks

La spécification BPMN2 propose un type de nœud spécifique appelé ***Business Rule Task***

Il est utilisé conjointement avec l'attribut de règle ***ruleflow-group.***

- Cette propriété permet de spécifier quelles règles peuvent être déclenchées lorsque le processus atteint le nœud du même nom

Exemple



Drools Fusion et CEP

Introduction

Le ***Complex event processing (CEP)*** permet de prendre des décisions basées sur des relations temporelles entre les faits.

L'objectif principal du CEP est de corréler de petites unités de données temporelles dans un nuage de données en constante évolution et en constante évolution afin de réagir à des situations spéciales difficiles à trouver.

Le raisonnement se fait sur des événements qui sont des faits avec une heure d'occurrence

Caractéristiques

- En général, il faut traiter de nombreux événements mais seulement un petit pourcentage a un réel intérêt.
- Les événements sont généralement immuables, (on ne peut pas changer le passé !).
- Les règles et les requêtes travaillent sur des pattern d’événements
- Il y a de fortes relations temporelles entre les événements

. Les événements pris isolément n'ont en général peu d'importance. Le système doit détecter des patterns d’événements reliés

- Le système doit composer et agréger les événements

Événement complexe

Un **complex event** est simplement une agrégation, une composition ou une abstraction d'autres événements

Les règles seront exprimées via des événements complexes utilisant l'agrégation, la composition ou l'abstraction

Sémantique des événements

2 types d'événements sont pris en compte :

- **Ponctuel**
- **Intervalle**

Tous les événements sont :

- Immuables
- Un cycle de vie géré: lorsqu'ils sont trop vieux, ils sont retirés de la session

Declaration des événements temporels

Afin de créer les règles CEP, il faut spécifier les types d'objets devant être traités comme des événements

Cela se fait en ajoutant des méta-données:

- ***@Role*** : *Fact* ou *Event*
- ***@Timestamp*** : L'attribut donnant l'estampillage si absent, le moment de l'insertion
- ***@Duration*** : L'attribut donnant la durée de l'événement (Optionnel)
- ***@Expires*** : L'attribut indiquant la date d'expiration de l'événement

Sample

```
@org.kie.api.definition.type.Role(Role.Type.EVENT)
@org.kie.api.definition.type.Duration("durationAttr")
@org.kie.api.definition.type.Timestamp("executionTime")
@org.kie.api.definition.type.Expires("2h30m")
public class TransactionEvent implements Serializable {
  private Date executionTime;
  private Long durationAttr;
  /* class content skipped */
}
```

Sample (2)

```
declare PhoneCallEvent
  @role(event)
  @timestamp(whenDidWeReceiveTheCall)
  @duration(howLongWasTheCall)
  @expires(2h30m)
  whenDidWeReceiveTheCall: Date
  howLongWasTheCall: Long
  callInfo: String
end
```

Opérateurs ternaires

Il y a 13 opérateurs temporels disponibles qui permettent de corréler les événements

Par exemple :

```
declare MyEvent
@role(event)
@timestamp(executionTime)
End

rule "my first time operators example"
  when
    $e1: MyEvent()
    $e2: MyEvent(this after[5m] $e1)
  Then
    System.out.println("We have two events" + " 5 minutes apart");
end
```

Exemple : délai d'activation

```
rule "Sound the alarm"
when
$f : FireDetected( )
not( SprinklerActivated( this after[0s,10s] $f ) )
then
// sound the alarm
end
```

Operator		Point - Point	Point - Interval	Interval - Interval
A before B	A			
	B			
A after B	A			
	B			
A coincides B	A			
	B			
A overlaps B	A			
	B			
A finishes B	A			
	B			
A includes B	A			
	B			
A starts B	A			
	B			
A finishedby B	A			
	B			
A startedby B	A			
	B			
A during B	A			
	B			
A meets B	A			
	B			
A metby B	A			
	B			
A overlappedby B	A			
	B			

Exemple d'agrégation

```
rule "Sound the alarm in case temperature rises above
   threshold"

when

TemperatureThreshold( $max : max )

Number( doubleValue > $max ) from accumulate(

SensorReading( $temp : temperature ) over window:time( 10m ),

average( $temp ) )

then

// sound the alarm

end
```

Autre exemple

```
rule "More than 10 transactions in an hour from one client"
  when
    $t1: TransactionEvent($cId: customerId)
    Number(intValue >= 10) from accumulate(
      $t2: TransactionEvent(this != $t1,
      customerId == $cId, this meets[1h] $t1),
      count($t2) )
    not (SuspiciousCustomerEvent(customerId == $cId,
      reason == "Many transactions"))
  then
    insert(new SuspiciousCustomerEvent($cId,
      "Many transactions"));
end
```

Cycle de vie des événements

- L'exécution en mode STREAM permet au moteur de détecter les événements qui ne peuvent plus matcher aucune règle et donc de les supprimer de la mémoire de travail
- L'identification des événements à supprimer se fait :
  - En se basant sur leur attribut d'expiration
  - En analysant les contraintes temporelles des règles

Fenêtres glissantes

Les **fenêtres glissantes** permettent de grouper des événements dans une fenêtre temporelle glissante.

Les fenêtres peuvent être exprimées via :

Un **temps** : Prise en compte des événements survenus lors des x dernières unités de temps

```
StockTick() over window:time( 2m )
```

Une **longueur** : Prise en compte des x derniers événements

```
StockTick( company == "IBM" ) over window:length( 10 )
```

Les fenêtre peuvent être définies :

- A l'intérieur d'une règle
- Ou à l'extérieur pour la réutilisation

Exemple basé sur la longueur

```
rule "last 6 transactions are more than 100 dollars"
When
  Number(doubleValue > 100.00) from accumulate(
    TransactionEvent($amount: totalAmount)
      over window:length(6),
    sum($amount))
then
  //... TBD
end
```

Exemple basé sur le temps

```
rule "obtain last five hours of operations"
when
  $n: Number() from accumulate(
   TransactionEvent($a: totalAmount)
     over window:time(5h),
   sum($a)
  )
Then
System.out.println("total = " + $n);
end
```

Déclaration de fenêtre glissante

```
declare window Beats
  @doc("last 10 seconds heart beats")
  HeartBeat() over window:time( 10s )
    from entry-point "heart beat monitor"
end

rule "beats in the window"
  when
    accumulate(HeartBeat() from window Beats,
      $cnt : count(1))
  then
   // there has been $cnt beats over the last 10s
end
```

Mise en place

Les Bases de connaissance et les Session nécessite une configuration spécifique :

- Kie Base doivent utiliser le mode ***stream***
- Il faut choisir sur le mode de déclenchement des règles : *discret* ou *continu*
- L'horloge interne de la KieSession doit être positionnée

KieBase

Kie Base doit être configurée pour utiliser le **STREAM** event processing mode.

Cette configuration informe le runtime qu'il doit gérer des événements et de les grader en interne ordonné via leur timestamp

Avec *kmodule.xml*

```
<kbase name="cepKbase" eventProcessingMode="stream"
packages="chapter06.cep">
<ksession name="cepKsession"/>
</kbase>
```

Discrete vs Continous

Le déclenchement des règles peut se faire :

- A des moments spécifiques.
  Après l'insertion d'un fait, on peut appeler *fireAllRules*
- Continuellement avec *fireUntilHalt*

Si l'absence d'événements doit déclencher une règle , il faut utiliser la façon continue

Si la seule chose pouvant déclencher une règle est l'insertion d'un fait, la méthode discrète est suffisante

Horloge de la session

- Par défaut, Kie Sessions utilise l’horloge de la machine sur laquelle elle s’exécute
- Pour les tests, on peut utiliser une pseudo-clock
- Pour des scénarios distribués, on peut utiliser une horloge synchronisée

Exemple Pseudo-clock

Configuration :

```
<kbase name="cepKbase" eventProcessingMode="stream"
packages="chapter06.cep">
  <ksession name="cepKsession" clockType="pseudo"/>
</kbase>
```

Usage dans un cas de test :

```
SessionPseudoClock clock = ksession.
getSessionClock();
clock.advanceTime(2, TimeUnit.HOURS);
clock.advanceTime(5, TimeUnit.MINUTES);
```

Alternatives au DRL

DSL
Tables de décision
Gabarits de règles

DSL

Introduction

Les ***Domain Specific Languages*** (DSL) permettent d'étendre le langage de règles en l'adaptant au langage métier.

C'est une couche d'abstraction dédiée aux experts métier non technique qui est traduite dans le langage de règle au moment de la compilation

Ils peuvent également être utilisés comme gabarits de conditions ou d'action, permettant ainsi de mutualiser certaines parties de règles

Syntaxe

Le format d'un DSL est tout simplement un fichier texte qui traduit des clés en langage « *naturel* » en des expressions *drl*

Chaque ligne commence par indiquer un ***scope***, puis la traduction du langage étendu dans le langage de règle

```
[when]This is "{something}"=Something(something=="{something}")"

[then]Log "{message}"=System.out.println("{message}") ;
```

Il est possible d'utiliser également le scope ***[keyword]*** qui permet de redéfinir un mot-clé :

```
[keyword] quand = when
```

Les phrases définies sont en fait des expressions régulières. Les wildcards peuvent donc être utilisés.

Exemples

```
[when]There is a Person with name of
  "{name}"=Person(name=="{name}")

[when]Person is at least {age} years old and lives in
  "{location}"=Person(age > {age}, location=="{location}")

[then]Log "{message}"=System.out.println("{message}");
```

```
There is a Person with name of "kitty"
  ---> Person(name="kitty")

Person is at least 42 years old and lives in "atlanta"
  ---> Person(age > 42, location="atlanta")

Log "boo"
  ---> System.out.println("boo");
```

Exemple avec regexp

```
[when][]is less than or equal to=<=
[when][]is less than=<
[when][]is greater than or equal to=>=
[when][]is greater than=>
[when][]is equal to===
[when][]equals===
[when][]There is a Cheese with=Cheese()
[when][]- {field:\w*} {operator} {value:\d*}={field} {operator} {value}
--------------------------------------------------------------------
There is a Cheese with
- age is less than 42
- rating is greater than 50
- type equals 'stilton'
--------------------------------------------------------------------
Cheese(age<42, type=='stilton', rating>50)
```

Mise en place

1. Nommer son fichier DRL avec l'extension *.dslr*

2. Faire référence au fichier DSL dans le fichier de règle avec le mot-clé *expander*

```
expander your-expander.dsl
```

3. Passer le DSL au moment de la compilation (Drools 5) :

```
PackageBuilder builder = new PackageBuilder() ;
builder.addPackageFromDrl( sourceReader,
dslReader );
```

Tables de décision

Présentation

Les tables de décisions sont un moyen efficace et compact pour représenter de la logique conditionnelle, elles sont adaptées aux experts métier

Les données saisies dans un tableur permettent de générer les règles.
=> l'expert métier bénéficie alors de son outil préféré : *Excel*

Pour chaque ligne de la table décisionnelle, les données sont combinées avec un gabarit pour générer une règle.

Les tables décisionnelles permettent d'encapsuler les règles et d'isoler le modèle objet. Seuls les paramètres des règles pouvant être modifiés sont exposés.

Exemple

	B	C	D	E
7				
8				
9		RuleSet	Some business rules	
10		Import	org.drools.decisiontable.Cheese, org.drools.deci	
11		Sequential	true	
12				
13		RuleTable Cheese fans		
14		CONDITION	CONDITION	ACTION
15		Person	Cheese	list
16	(descriptions)	age	type	add("$param")
17	Case	Persons age	Cheese type	Log
18	Old guy	42	stilton	Old man stilton
19	Young guy	21	cheddar	Young man cheddar
20				
21		Variables	java.util.List list	

Syntaxe des gabarits

Les tables décisionnelles comportent 2 types de colonnes :

- Colonnes de **condition** <=> LHS, la syntaxe de contrainte doit être utilisée
- Colonnes **d'action** <=> RHS, la syntaxe de code doit être utilisée

$param est utilisé pour indiquer ou seront insérées les données des cellules (*$1* peut être utilisé)

Si la cellule contient une liste de valeur séparées par des virgules les symboles *$1*, *$2*, etc. peuvent être utilisés.

La fonction *forall(DELIMITER){SNIPPET}* peut être utilisée pour effectuer une boucle sur toutes les valeurs disponibles.

Condition

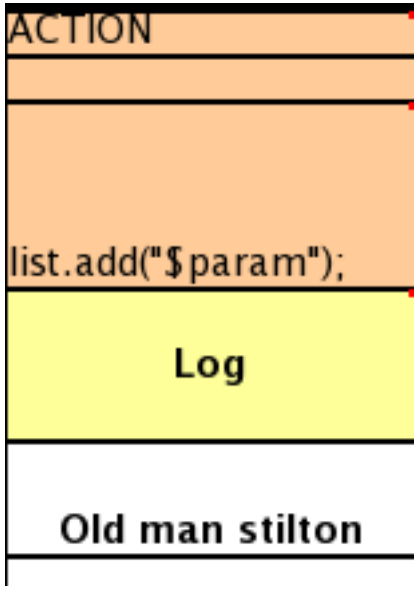
Le rendu d'une condition dépend de la présence d'une déclaration d'un type d'objet dans une ligne au-dessus. Si le type est précisé, une contrainte de type est créée. Si la cellule contient juste un attribut, la contrainte sera une contrainte d'égalité, sinon la cellule inclus un opérateur .

13	RuleTable Cheese fans	
14	CONDITION	CONDITION
15	Person	
16	age	type
17	Persons age	Cheese type
18	42	stilton

Conséquences

Le résultat d'une cellule action dépend de la présence d'une entrée sur la ligne immédiatement au dessus.

- Si il n'y a rien, la cellule est interprétée telle quelle
- Si il y a une variable, le contenu de la cellule est ajouté à la variable (Appel de méthode)



Mots-clés

Avant le mot-clé *RuleTable*, les mots-clés suivants peuvent être présent et conditionnent leur cellule immédiatement à droite:

- ***RuleSet*** : Spécification du nom du groupe de règles (package), si vide c'est le groupe par défaut
- ***Sequential*** : La cellule à droite contient *true* ou *false*. Si *true*, la propriété *salience* est utilisé pour garantir l'ordre
- ***Import*** : Liste de classes java à importer
- ***Functions*** : Déclaration de fonctions
- ***Variables*** : Déclaration de variables globales
- ***Queries*** : Déclaration de requêtes

Exemple

RuleSet	Control Cajas[1]
Import	foo.Bar, bar.Baz
Variables	Parameters parametros, RulesResult resultado, EvalDate fecha

Functions	function boolean isRango(int iValor, int iRangoInicio, int iRangoFinal) {
	if (iRangoInicio <= iValor && iValor <= iRangoFinal)
	return true;
	return false;
	}
	
	function boolean isIgualTipo(TipoVO tipoVO, int p_tipo, boolean isNull) {
	if (tipoVO == null)
	return isNull;
	return tipoVO.getSecuencia().intValue() == p_tipo;
	}

RuleTable

Une cellule avec ***RuleTable*** indique le début de la définition d'une table de règle.

La table démarre avec la ligne suivante. Elle est lue de gauche à droite et de bas en haut jusqu'à une ligne blanche.

Mots-clés dans la table de règles

CONDITION : Indique une colonne condition

ACTION : Indique une colonne action

PRIORITY : Indique une colonne utilisé pour l'attribut *salience*

DURATION : Indique l'attribut *duration* de la règle

NAME : Le nom de la règle (optionnel)

NO-LOOP : Attribut *no-loop* (*true* ou *false*)

ACTIVATION-GROUP : Attribut *activation-group*

AGENDA-GROUP : Attribut *agenda-group* de la règle

RULEFLOW-GROUP : Attribut *ruleflow-group* de la règle

Intégration

- L'intégration d'une table décisionnelle nécessite la librairie ***drools-decisiontables.jar***
- La classe principale ***SpreadsheetCompiler*** prend en entrée un fichier *csv* ou *excel* et génère les règles en DRL
- Les règles peuvent être alors manipulées indépendamment

Scénario de mise en place

1. L'expert métier démarre à partir d'un gabarit de table décisionnelle
2. Il renseigne les paramètres des règles et les actions avec des descriptions métier
3. Ils saisit les lignes correspondant aux règles
4. La table décisionnelle est reprise par un technicien qui fait correspondre le langage métier à des scripts
5. L'expert métier et le technicien revoient ensemble les modifications effectuées.
6. L'expert métier peut éditer les règles selon ses besoins.
7. Le technicien peut écrire des cas de tests vérifiant les règles

Gabarits de règles

Gabarits de règle

Les **gabarits de règle** utilisent des sources de données tabulaires (Tableur, CSV ou autres) pour générer de nombreuses règles.

C'est une technique finalement plus puissante que les tables de décisions :

- Les données peuvent être stockées dans une base de données
- La génération de règle peut être conditionnée par les données
- Les données peuvent être utilisées dans n'importe quelle partie des règles (opérateur, nom d'une classe, nom d'une propriété)
- Plusieurs gabarits peuvent être exécutées sur les mêmes données

Structure gabarit

Le fichier texte

- commence par une entête ***template header*** .
- Ensuite, la liste des colonnes des données tabulaires
- Une ligne vide pour marquer la fin des définitions de colonne
- L'entête standard DRL (package, import, global, fonctions)
- Le mot clé ***template*** marque le début d'un gabarit de règle, il peut en avoir plusieurs dans le fichier.
- Le gabarit utilise la syntaxe ***@{token_name}*** pour les substitutions (ex : *@{row.rowNumber}*
- Le mot clé ***end template*** marque la fin du gabarit.

Exemple

```
template header
age
type
Log

package org.drools.examples.templates;

global java.util.List list;

template "cheesefans"

rule "Cheese fans_@{row.rowNumber}"
when
Person(age == @{age})
Cheese(type == "@{type}")
then
list.add("@{log}");
end
end template
```

kmodule

Le gabarit doit ensuite être inclus avec le fichier de donnée associés dans la définition du *kmodule*

```
<?xml version="1.0" encoding="UTF-8"?>
<kmodule xmlns="http://drools.org/xsd/kmodule">
<kbase name="TemplatesKB" packages="org.drools.examples.templates">
<ruleTemplate
  dtable="org/drools/examples/templates/ExampleCheese.xls"
template="org/drools/examples/templates/Cheese.drt"
row="2" col="2"/>
<ksession name="TemplatesKS"/>
</kbase>
</kmodule>
```

Exemple BD

```
// Get results from your DB query...
resultSet = preparedStmt.executeQuery();
// Generate the DRL...
resultSetGenerator = new ResultSetGenerator();
String drl = resultSetGenerator.compile(resultSet,
        new
  FileInputStream("path/to/template.drt"));
```

Intégrer Kie avec les applications

Considérations
Patterns d’intégration
Dépôts de connaissances Maven
KAAS
KieWorkbench

Communication entre le moteur et l'application cliente

Différents mécanismes peuvent être utilisés :

- Les variables globales
- Les Entry points : Partionnement de faits ou de flux d'événements
- Channels pour envoyer des informations au monde extérieur
- Listeners ou marshallers (sérialisation de session)

Mode synchrone

Si nous devons exécuter nos règles métier de manière synchrone

- Utiliser des KieSessions stateless et créer autant de sessions que de requêtes
- Utiliser des variables globales pour stocker des informations d'exécution de règles

Mode asynchrone

Les sessions Kie peuvent être partagées entre différents threads. Certaines d'entre elles peuvent insérer de nouvelles informations et d'autres peuvent prendre soin de déclencher des règles.

- Enregistrez des listener spécifiques qui veillent à informer les autres composants des situations spéciales détectées par les règles
- Les points d'entrée deviennent un composant très utile lorsque plusieurs sources insèrent des informations dans une seule session Kie

Patterns d’intégration

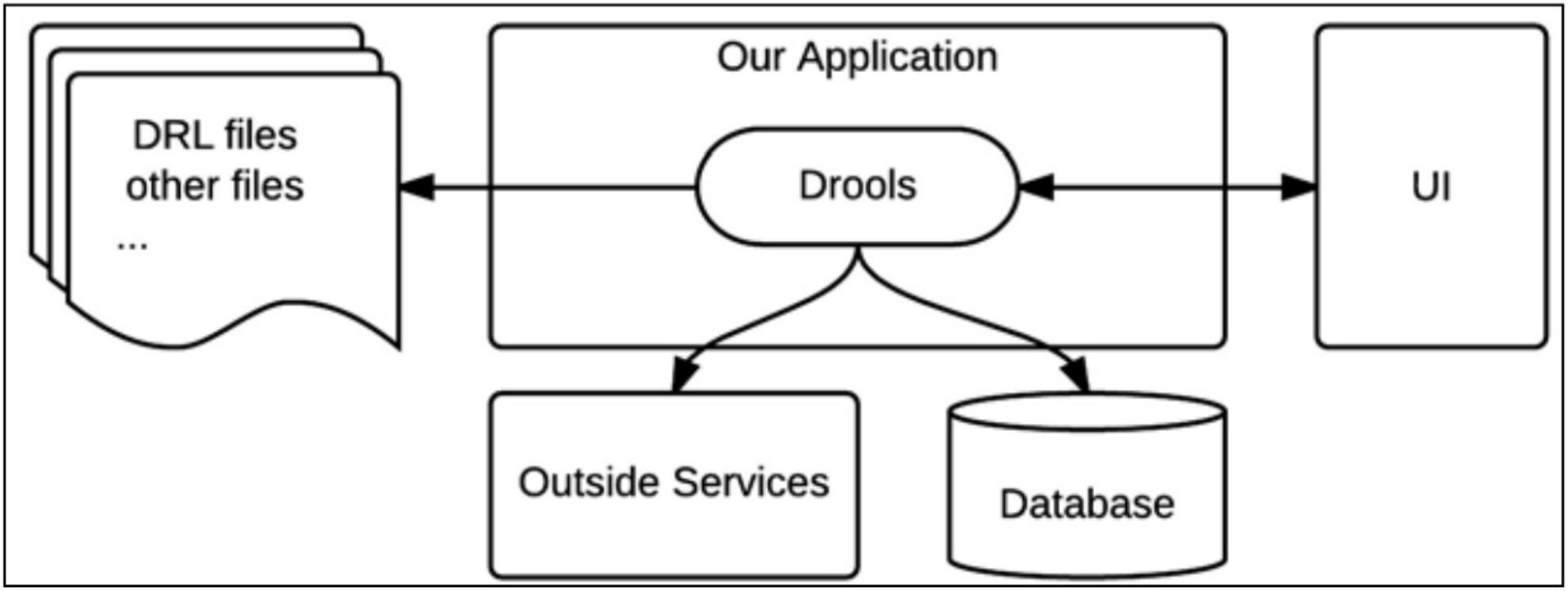
Scenarios d’intégration

Différentes mécanisme d'intégration peuvent être envisagés :

- Embarquer les règles et le runtime Drools dans l’application cliente
- Externaliser les règles dans un dépôt séparé
- Accéder au règle via un service web : Knowledge As A Service

Pour tous ces scénarios, Drools propose du support pour CDI, Spring et Camel

Architecture :
Drools et Ressources embarquées



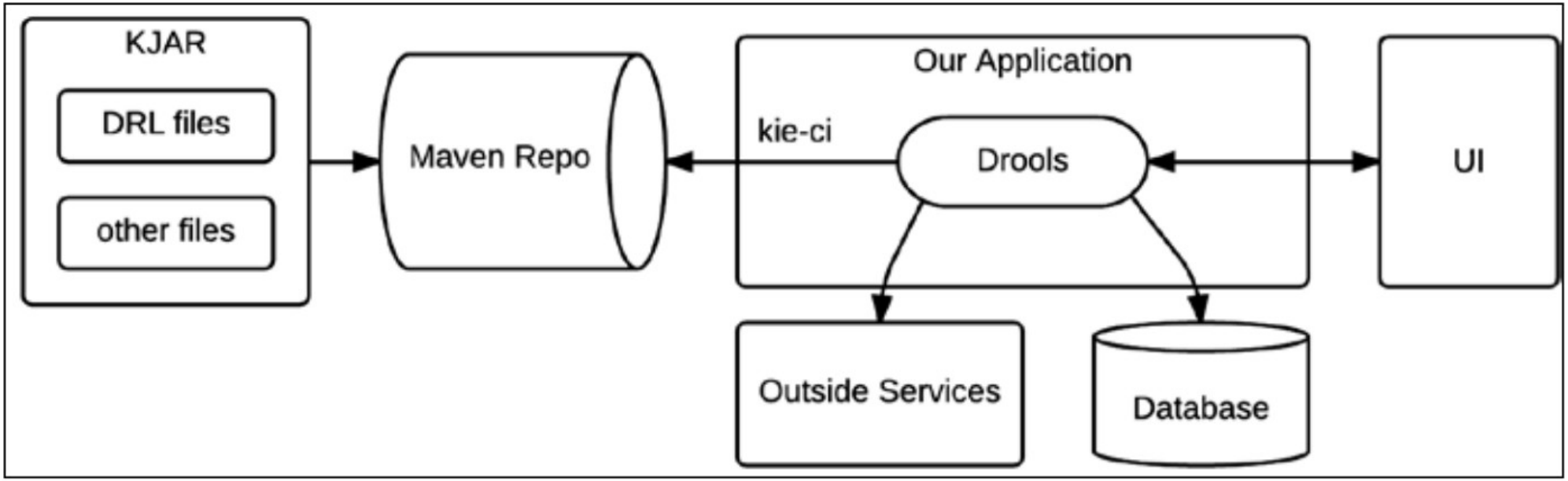
Repository Maven

Les règles peuvent être séparées du reste de l'application en utilisant un dépôt Maven :

- Les règles métier sont déployé dans le dépôt sous forme de KJAR
- Les règles peuvent être chargées dynamiquement dans l'application (qui contient le runtime Drools)

=> Les règles peuvent être développées, et déployées autant de fois que nécessaire sans avoir à redéployer les applications.

Architecture
Dépôt Maven



Chargement des règles

Avec CDI

```
@Inject @KSession
@KReleaseId(groupId = "org.drools.devguide",artifactId =
  "chapter-11-kjar",
version = "0.1-SNAPSHOT")
KieSession kSession;
```

Avec un scanner

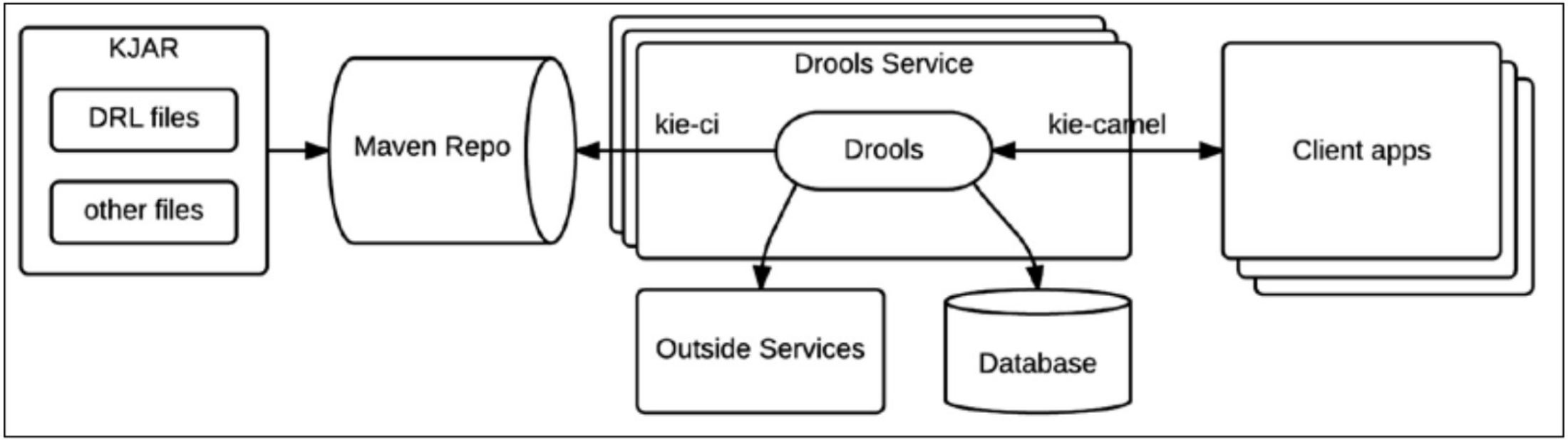
```
KieServices ks = KieServices.Factory.get();
KieContainer kContainer = ks.newKieContainer(
ks.newReleaseId("org.drools.devguide",
"chapter-11-kjar", "0.1-SNAPSHOT"));
KieScanner kScanner = ks.newKieScanner(kContainer);
kScanner.start(10_000);
KieSession kSession = kContainer.newKieSession();
```

Knowledge As A Service

La dernière approche consiste à externaliser le runtime Drools dans un service externe.

- Ce service expose une API (REST ou autre)
- Plusieurs applications peuvent accéder au service
- Le service peut être répliqué si nécessaire
- Le cycle de vie des règles est complètement indépendant des applications clientes

Architecture
KAAS



Dépôt de connaissance Maven

Construire un kjar
Mise à jour des règles métier

Introduction

Un des premiers objectifs lors de l'utilisation d'un moteur de règles est de décorréler les règles métier de l'application qui l'utilise.

Drools s'appuie sur Maven pour déployer les règles métier dans un dépôt.

Les applications peuvent alors :

- Soit déclarer une dépendance Maven vers l'artefact des règles métier enfin de charger les règles via le *classpath*
- Soit accéder à un dépôt distant dynamiquement (http, Scanner)

Construction d’un *kjar* avec Maven

- Un projet « règles métier » est typiquement un projet Maven avec en plus un fichier *kmodule.xml* configurant les différentes bases de connaissances
- Le cycle de build est enrichi avec le plugin Kie qui précompile les règles et s'assurent de leur validité

```
<packaging>kjar</packaging>
...
<build>
  <plugins>
    <plugin>
      <groupId>org.kie</groupId>
      <artifactId>kie-maven-plugin</artifactId>
      <version>${project.version}</version>
      <extensions>true</extensions>
    </plugin>
  </plugins>
</build>
```

Configuration par défaut

kmodule.xml définit les bases de connaissances et les sessions pouvant être utilisées dans le projet

Un fichier vide donne une configuration par défaut :

- Une unique base de connaissance comprenant toutes les règles/processus/etc. trouvés dans le répertoire ***resources*** du jar
- Une session *stateful* et une session *stateless*

Exemple *kmodule.xml*

```
<kmodule xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance " xmlns="http://jboss.org/kie/6.0.0/kmodule">
  <kbase name="KBase1" default="true" eventProcessingMode="cloud" equalsBehavior="equality">
    <ksession name="KSession2_1" type="stateful" default="true/">
    <ksession name="KSession2_1" type="stateless" default="false" beliefSystem="jtms"/>
</kbase>
<kbase name=""KBase2" default="false" eventProcessingMode="stream" equalsBehavior="equality"
 declarativeAgenda="enabled"
packages="org.domain.pkg2,org.domain.pkg3" includes="KBase1">
<ksession name="KSession2_1" type="stateful" default="false" clockType="realtime">
  <fileLogger file="drools.log" threaded="true" interval="10"/>
  <workItemHandlers>
    <workItemHandler name="name" type="org.domain.WorkItemHandler"/>
  </workItemHandlers>
  <listeners>
    <ruleRuntimeEventListener type="org.domain.RuleRuntimeListener"/>
    <agendaEventListener type="org.domain.FirstAgendaListener"/>
    <agendaEventListener type="org.domain.SecondAgendaListener"/>
    <processEventListener type="org.domain.ProcessListener"/>
  </listeners>
</ksession>
</kbase>
</kmodule>
```

Attributs de *kbase*

nom	défaut	autorisé	description
name	aucune		Obligatoire. Utilisé pour récupérer la base à partir du conteneur
includes	aucune	Liste d'item séparés par des virgules	Les bases de connaissance à inclure
packages	all	Liste d'item séparés par des virgules	Liste les ressources à compiler
default	false	true/false	Il n'est pas nécessaire de préciser le nom pour la base de connaissance par défaut
equalsBehaviour	identity	identity/equality	Test de présence d'un fait dans la base de connaissance
eventProcessingMode	cloud	cloud/stream	Stream pour drools-fusion
declarativeAgenda	disabled	disabled/enabled	Agenda déclaratif activé ou non expérimental, les règles peuvent agir directement sur les activations présentes dans l'agenda

Attributs de *ksession*

nom	défaut	autorisé	description
name	aucun	N'importe	Obligatoire. Nom utilisé pour récupérer une session du conteneur
type	stateful	stateless/ stateful	
default	false	true/false	Il n'est pas nécessaire d'indiquer le nom pour la session par défaut
clockType	realtime	realtime/ pseudo	Indique si les timestamp des événements est fourni par le système ou par l'application
beliefSystem	simple	simple/jtms/ defeasible	Système de gestion des faits

Éléments de ksession

Un élément <*ksession*> peut également définir différents sous-éléments :

- Un file ***logger*** permet de définir un fichier de trace qui consignera tous les évènements Drools dans un fichier
- Des ***WorkItemHandlers*** permet de définir des gestionnaires de tâche qui sont associés à des nœuds spécifiques de Drools-flow (ex : Tâche humaine)
- Des ***listeners*** d'événements Drools : *ruleRuntimeEventListener*, *agendaEventListener* ou *processEventListener*

Classpath Container (Drools6)

Une fois que l'application a accès à *kmodule.xml*, elle peut récupérer les bases et créer des sessions via leur nom.

```
KieServices kieServices = KieServices.Factory.get();
KieContainer kContainer = kieServices.getKieClasspathContainer();
KieBase kBase1 = kContainer.getKieBase("KBase1");
KieSession kieSession1 = kContainer.newKieSession("KSession2_1");
StatelessKieSession kieSession2 = kContainer.newStatelessKieSession("KSession2_2");
```

Maven et Release ID (Drools6)

Si le projet KIE est un projet Maven, il est possible d'utiliser les coordonnées Maven pour instancier le conteneur.

```
KieServices kieServices = KieServices.Factory.get();
ReleaseId releaseId = kieServices.newReleaseId( "org.acme", "project", "1.0" );
KieContainer kieContainer = kieServices.newKieContainer( releaseId ) ;
```

Style Drools 7

Si nous préférons utiliser *RuntimeManager*, l'interface *RuntimeEnvironmentBuilderFactory* fournit des méthodes basées sur classpath ou *ReleaseId* :

```
// Based on classpath
public RuntimeEnvironmentBuilder
  newClasspathKmoduleDefaultBuilder();
// Based on Maven repository
public RuntimeEnvironmentBuilder newDefaultBuilder(String
  groupId, String artifactId, String version);
public RuntimeEnvironmentBuilder newDefaultBuilder(ReleaseId
  releaseId);
```

Exemple

```
ReleaseId releaseId = ks.newReleaseId("org.jbpm", "helloworld", "1.0");
builder = RuntimeEnvironmentBuilder.Factory.get()
          .newDefaultBuilder(releaseId)
            .entityManagerFactory(emf)
            .registerableItemsFactory(new DefaultRegisterableItemsFactory() {
                 public Map<String, WorkItemHandler> getWorkItemHandlers(RuntimeEngine runtime) {
                     return myhandlers;
                 }
              public List<ProcessEventListener> getProcessEventListeners(RuntimeEngine runtime) {
                     return myProcessListeners;
                 }
              public List<AgendaEventListener> getAgendaEventListeners( RuntimeEngine runtime) {
                     return myAgendaListeners;
                   }
              public List<TaskLifeCycleEventListener> getTaskListeners() {
                     return myTaskListener;
              }
            })
            .userGroupCallback(userGroupCallback)
```

Mise à jour des règles

Déploiement

Le déploiement consiste à mettre à jour le dépot Maven.

L'application cliente peut alors:

- Être mise à jour manuellement pour utiliser une nouvelle version du kjar
- Être dépendante de la dernière version
- Être dépendante d'une famille de version

KieScanner

KieScanner permet de scruter en permanence le repository Maven afin de vérifier si une nouvelle release du projet est disponible

Dans ce cas, la nouvelle release est déployée dans le *KieContainer* et les nouvelles règles sont prises en compte

L'utilisation de *KieScanner* nécessite la présence de ***kie-ci.jar*** dans le classpath

Enregistrement du scanner

```
KieServices kieServices = KieServices.Factory.get();

ReleaseId releaseId = kieServices.newReleaseId( "org.acme",
   "myartifact", "1.0-SNAPSHOT" );

KieContainer kContainer =
   kieServices.newKieContainer( releaseId );

KieScanner kScanner =
   kieServices.newKieScanner( kContainer );

// Démarrage de KieScanner

// qui scrute le repository toutes les 10 secondes

kScanner.start( 10000L );
```

Mise à jour

La mise à jour automatique est effective si :

- la version de l'artifact est suffixée avec SNAPSHOT, LATEST RELEASE ou un intervalle de version
- le *KieScanner* trouve une mise à jour dans le dépôt Maven

La mise à jour consiste à télécharger la nouvelle version et d'effectuer un build incrémental. Les KieBases et KieSessions contrôlé par le KieContainer sont automatiquement mises à jour et toutes les nouvelles KieBases ou KieSessions utilisent la nouvelle version du projet.

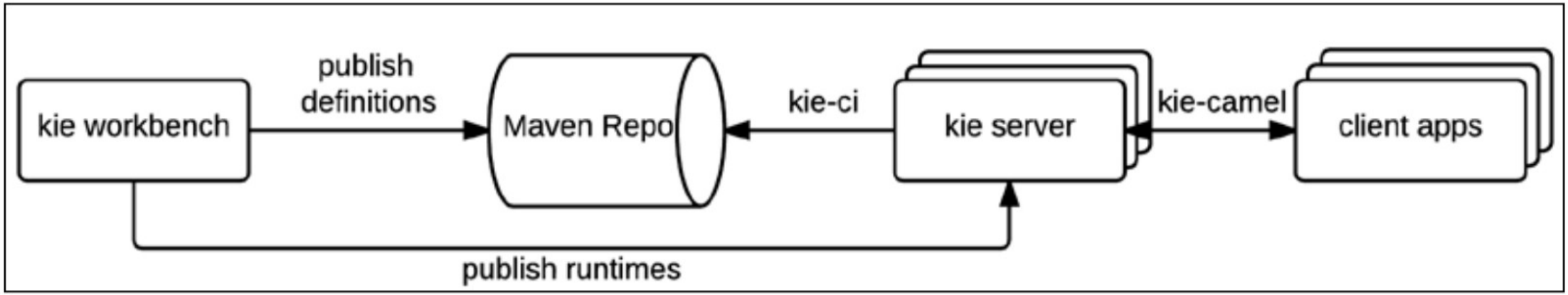
Knowledge As A Service

Introduction

KIE fournit un module appelé ***kie-server***, qui permet de configurer des environnements similaires uniquement responsable de l'exécution des règles et des processus métier, en prenant le kJAR de sources externes

Il fournit également ***kie-workbench*** qui offre une interface utilisateur pour gérer les règles et les processus, construire les bases de connaissance et les déployer sur les environnements KieServer

Architecture



Kie Server

Composant serveur autonome et modulaire utilisé pour exécuter des règles et des processus, packagé sous forme de WAR

- Disponible pour les conteneurs Web et les conteneurs d'applications JEE6 et JEE7
- Peut être facilement déployé dans des environnements cloud
- Chaque instance de serveur peut gérer de nombreux conteneurs Kie
- Mécanisme d’extension via *Kie Server Extensions*

Mise en place

Distributions : archive zip ou Image docker

Utilise un serveur jBoss wildfly dans la configuration full.

Définir un utilisateur avec le rôle *kie-server*

L'API REST est alors disponible à :
http://localhost:8080/kie-server/services/rest/server/

Une documentation swagger est disponible à :

http://localhost:8080/kie-server/docs/

Exemple de réponse

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<response type="SUCCESS" msg="Kie Server info">
    <kie-server-info>
        <capabilities>KieServer</capabilities>
        <capabilities>BRM</capabilities>
        <capabilities>BPM</capabilities>
        <capabilities>CaseMgmt</capabilities>
        <capabilities>BPM-UI</capabilities>
        <capabilities>BRP</capabilities>
        <capabilities>DMN</capabilities>
        <capabilities>Swagger</capabilities>
        <location>http://localhost:8230/kie-server/services/rest/server</location>
        ...
    </kie-server-info>
</response>
```

Client REST Natif

L'interaction avec le *kie-server* peut être effectuée par n'importe quel client REST (curl, postman, Spring RestTemplate,...)

Mais KIE propose un client natif adapté à ce type d'interaction.

Dépendance Maven :
`org.kie:kie-server-client`

Configuration d’une requête cliente

La configuration du client comprend:

- Informations d'identification de l'utilisateur *kie-server*
- Emplacement du serveur KIE,
  Ex : *http://localhost:8080/kie-server/services/rest/server*
- Format de marshaling pour les demandes et réponses API (JSON, JAXB ou XSTREAM)
- Les objets *KieServicesConfiguration* et *KieServicesClient*, qui servent de point d'entrée pour démarrer la communication avec le serveur
- Un objet *KieServicesFactory* définissant le protocole REST et l'accès utilisateur
- Tout autre service client utilisé, tel que *RuleServicesClient*, *ProcessServicesClient* ou *QueryServicesClient*

Types de client

Il existe différents types de clients, chacun étant spécialisé dans un aspect de KIE:

- ***RuleServicesClient*** : Utilisé pour insérer / enlever des faits et pour déclencher les règles
- ***ProcessServicesClient*** : Utilisé pour démarrer, signaler annuler des processus ou des tâches du processus
- ***QueryServicesClient***: Interrogation des processus
- ***UserTaskServicesClient*** : Exécution des tâches d'un processus
- ...

Exemple (1)

```
public static void main(String[] args) {
        initializeKieServerClient();
        initializeDroolsServiceClients();
        initializeJbpmServiceClients();
    }
```

Exemple (2)

```
public static void initializeKieServerClient() {
        conf = KieServicesFactory.newRestConfiguration(URL, USER,
   PASSWORD);
        conf.setMarshallingFormat(FORMAT);
        kieServicesClient =
   KieServicesFactory.newKieServicesClient(conf);
    }


    public static void initializeDroolsServiceClients() {
        ruleClient =
   kieServicesClient.getServicesClient(RuleServicesClient.class);
        dmnClient =
   kieServicesClient.getServicesClient(DMNServicesClient.class);
    }
```

Exemple (3)

```
public static void initializeJbpmServiceClients() {
        caseClient =
   kieServicesClient.getServicesClient(CaseServicesClient.class);
        documentClient =
   kieServicesClient.getServicesClient(DocumentServicesClient.class);
        jobClient =
   kieServicesClient.getServicesClient(JobServicesClient.class);
        processClient =
   kieServicesClient.getServicesClient(ProcessServicesClient.class);
        queryClient =
   kieServicesClient.getServicesClient(QueryServicesClient.class);
        uiClient = kieServicesClient.getServicesClient(UIServicesClient.class);
        userTaskClient =
   kieServicesClient.getServicesClient(UserTaskServicesClient.class);
    }
```

Exemple
Utilisation de RuleService

```
public void executeCommands() {

  String containerId = "hello";
  System.out.println("== Sending commands to the server ==");
  RuleServicesClient rulesClient = kieServicesClient.getServicesClient(RuleServicesClient.class);
  KieCommands commandsFactory = KieServices.Factory.get().getCommands();

  Command<?> insert = commandsFactory.newInsert("Some String OBJ");
  Command<?> fireAllRules = commandsFactory.newFireAllRules();
  Command<?> batchCommand = commandsFactory.newBatchExecution(Arrays.asList(insert, fireAllRules));

  ServiceResponse<String> executeResponse = rulesClient.executeCommands(containerId, batchCommand);

  if(executeResponse.getType() == ResponseType.SUCCESS) {
    System.out.println("Commands executed with success! Response: ");
    System.out.println(executeResponse.getResult());
  } else {
    System.out.println("Error executing rules. Message: ");
    System.out.println(executeResponse.getMsg());
  }
}
```

TP: Utilisation client natif

Kie workbenches

Workbenches

Les projets Kie fournissent des workbenches qui permettent de créer, construire et déployer des processus et/ou règles dans des serveurs Kie

- Ces applications sont construites avec le framework UberFire.
- Ce sont des applications web déployées sur un serveur d'applications (Jboss wildfly)

Installation

Télécharger la distribution ***kie-wb-*.war***

Deployer le war dans un serveur applicatif

- Drools 7 disponible uniquement pour Wildfly 14

L'application est disponible à:
http://<server>:<port>/

Données du workbench

- Les données internes du workbench sont stockées dans : ***$WORKING_DIRECTORY/.niogit*** , par exemple *wildfly-8.0.0.Final/bin/.niogit*
- C'est le répertoire à sauvegarder en production

Gestion des utilisateurs

Le WB utilise les rôles suivants :

- ***admin*** : Administrateur, gère les utilisateurs, les repositories, ...
- ***developer*** : Quasiment les mêmes droits qu'admin sauf le clonage de repositiory. Gérer les règles, les modèles, les processus, formulaires, …. Crée, construit et déploie les projets
- ***analyst*** : Idem *developer* avec droit restreint (pas de déploiement par exemple)
- ***user*** : Participe aux processus métier et réalise des tâches
- ***manager*** : Accède au reporting

Dans la configuration simple, les utilisateurs sont créés via *$JBOSS_HOME/bin/add-user.sh*

Démarrage

Actions typiques pour démarrer :

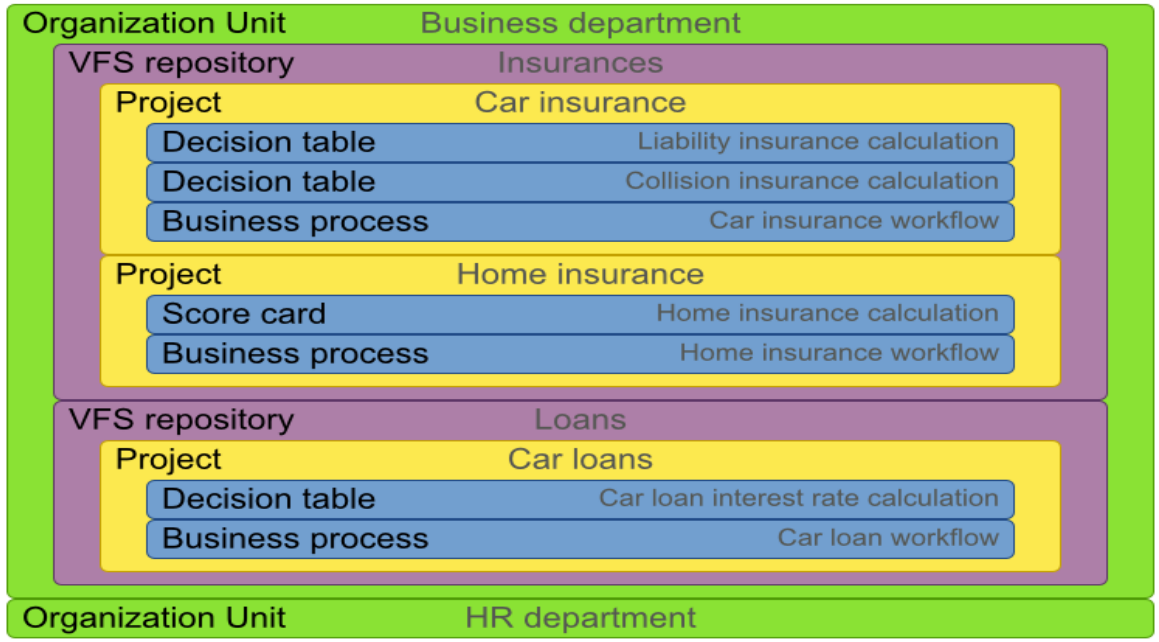
1. Créer un utilisateur avec le rôle admin
2. Ajouter un dépôt
3. Ajouter un projet
4. Fournir le modèle métier
5. Créer les règles
6. Construire et déployer

Ces actions peuvent être faites par l'interface web, par un outil de commande en ligne (*kie-config-cli.sh*) ou via une interface REST

Organisation

Un workbench est structuré avec des unités organisationnelles, des dépôts et des projets

Workbench structure overview



Explorateur projet

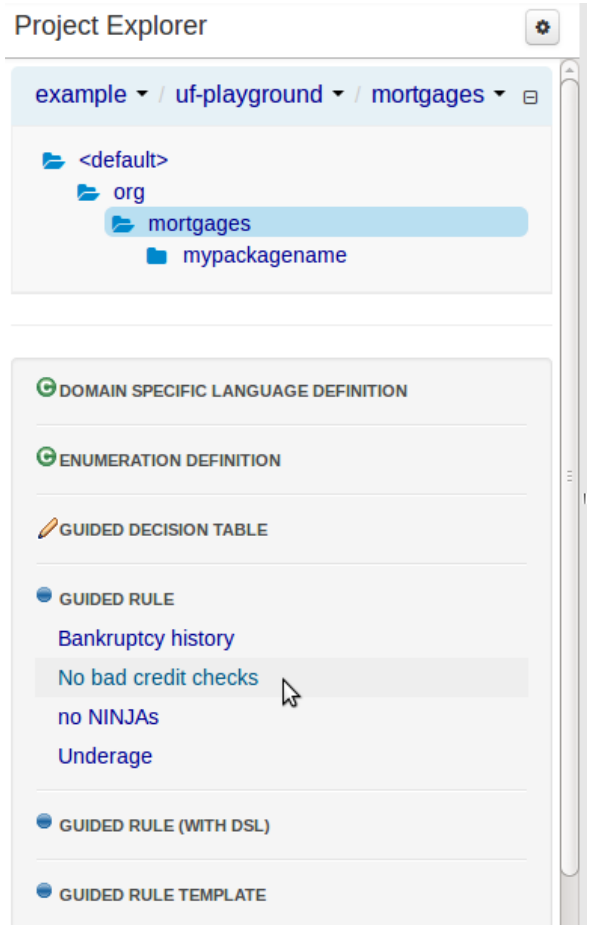
L'explorateur projet permet de parcourir les différentes unités organisationnelles, les dépôts, les projets et les ressources d'un projet

L'utilisateur, après avoir sélectionné un projet, peut accéder à ses packages et voir leur contenu

2 vues sont disponibles :

- Vue projet : certains fichiers sont cachés
- Vue dépôt: tous les fichiers sont affichés

Exemple


Project Explorer
example / uf-playground / mortgages
<default>
org
mortgages
mypackagename
DOMAIN SPECIFIC LANGUAGE DEFINITION
ENUMERATION DEFINITION
GUIDED DECISION TABLE
GUIDED RULE
Bankruptcy history
No bad credit checks
no NINJAs
Underage
GUIDED RULE (WITH DSL)
GUIDED RULE TEMPLATE

Éditeur projet

L'éditeur projet centralise toutes les configurations projet disséminés dans différents fichiers

- **Projet : *pom.xml***
  - Général : Coordonnées Maven
  - Dépendances
  - Méta-données
- **Base de connaissance : *kmodule.xml***
  - Bases de connaissance et sessions
  - Méta-données
- **Imports : *project.imports***
  - Suggestions : les imports suggérés dans l'éditeur d'assets
  - Méta-données

Le bouton Build & Depoy construit le projet et déploie le KJAR dans le dépôt Maven interne du workbench

Modèle de donnée

Il est possible de définir son modèle de donnée via Drools-WB

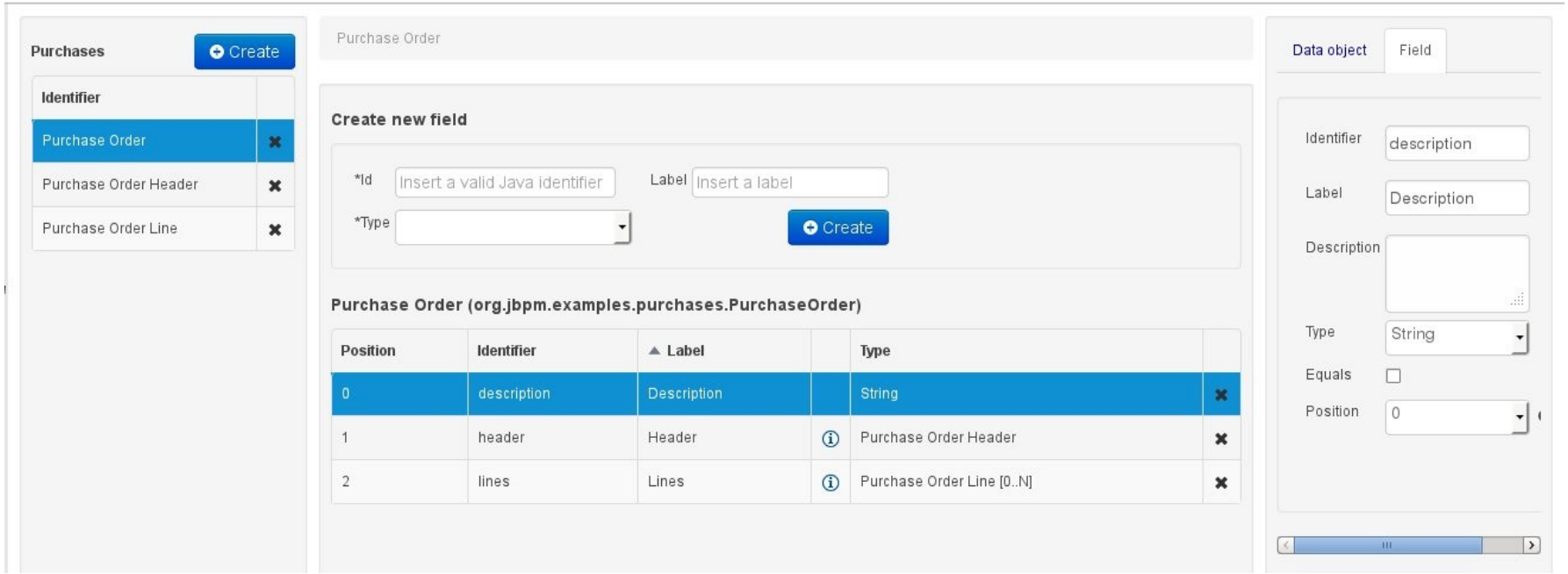
Un modèle de données est toujours associé à un projet

La vue ***Data Modeller tool*** permet de définir de nouvelles entités

Une entité contient :

- Des attributs modeleur : identifiant, label, description, package
- Des attributs applicatifs : types de base Java ou autre entité

Data Modeller



Génération de code

Le modeleur est un outil visuel qui lorsque le bouton « *Save* » est actionné, génère une classe Java POJO (Plain Old Java Object).

La classe contient un constructeur sans argument, des getters/setters pour ses attributs applicatifs, des constantes annotées pour les attributs *modeleur,* les méthodes *equals()* et *hashCode()*

Modèle externe

Les projets peuvent utiliser des modèles définies dans des classes Java créées à l'extérieur du workbench.

La dépendance vers le jar contenant les classes est indiquée :

- Soit par rapport au repository Maven local (Bouton *Add*)
- Soit par rapport au repository Guvnor M2 du workbench (Bouton *Add from repository*)

Asset

Drools-WB regroupe les différents ressources qu'il gère sous le terme **d'asset**.

Un *asset* est une ressource stockée et versionnée dans le repository

- Assets métier : Tables décisionnelles, Règles métier en DSL
- Assets techniques : Règles DRL, Énumérations, Processus.
- Fonctions :
- DSL: Domain Specific Languages
- Modèle : Les classes ou les types de faits du modèle métier
- WorkingSets : Sous-ensemble du modèle sur lequel des contraintes peuvent s'appliquer globalement.
- Des cas de test

Les *assets* peuvent être taggés.

Packages

La configuration des packages est généralement effectuée une seule fois par une personne ayant une expertise des règles et des modèles

- Un package est créé en spécifiant son nom
- Tout *asset* appartient à un seul package qui agit comme un espace de noms

Formats de règles

Le workbench prend en charge plusieurs formats de règles et éditeurs:

- Format BRL avec BRL Editor (également présent dans Eclipse)
- DSL
- Tables de décision dans le fichier à télécharger
- Tables de décision éditées directement dans l'interface Web
- Processus de flux de règles à télécharger
- DRL
- les fonctions
- Configuration des listes de valeurs (Enum)
- Gabarit de règles alimenté par des tableaux de données

Il est toujours possible de consulter la source en drl

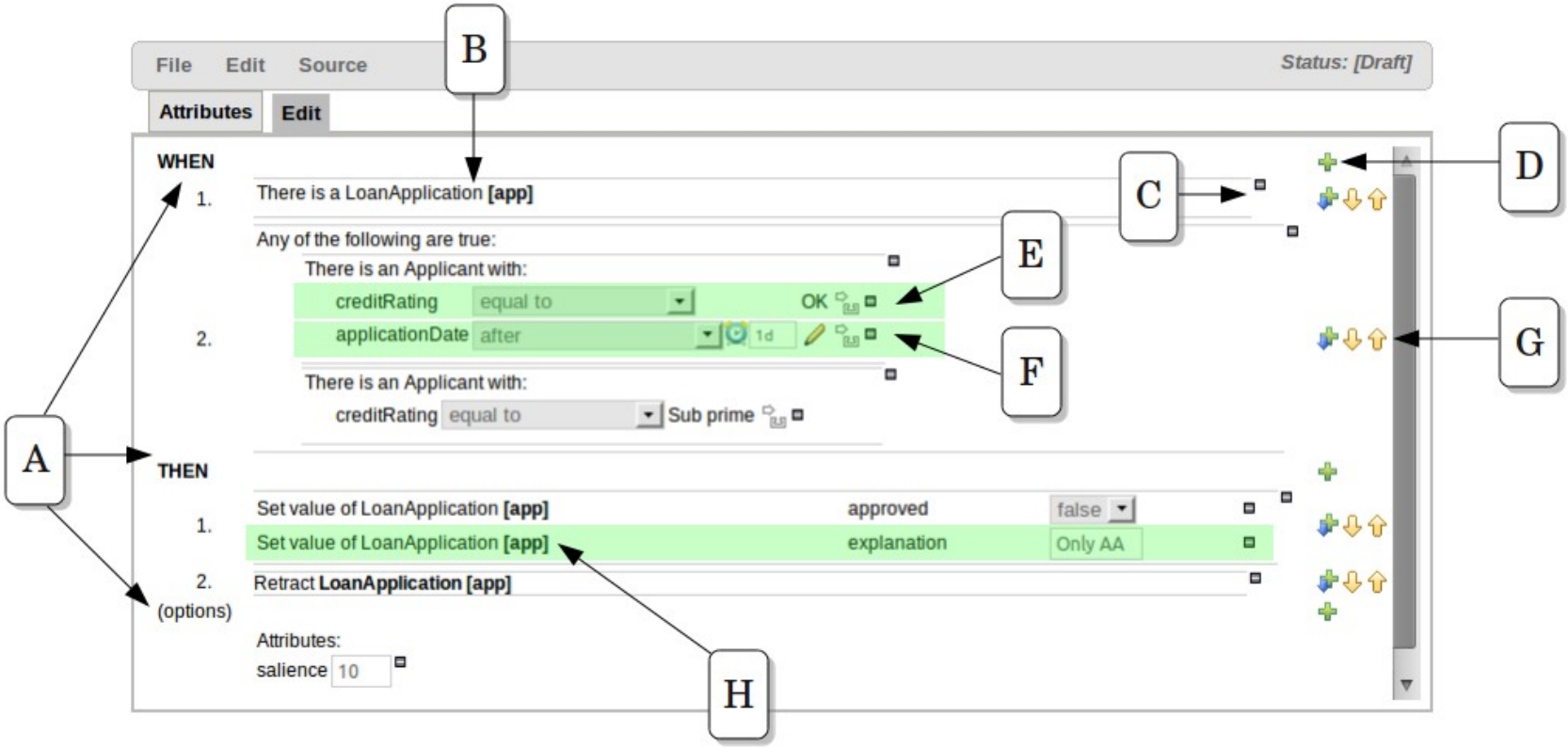
Nouvelle règle → nom, catégorie et format

=> Un assistant démarre

Validation

- Drools-WB permet de valider les règles créées
- Le panneau ***Problem*** affiche les validations en temps réel des différents assets
- Tous les éditeurs permettent une validation avant l'enregistrement du fichier

Éditeur guidé de règles métier



Éditeur guidé de gabarit

Guided Template [t1]

EXTENDS None selected

WHEN

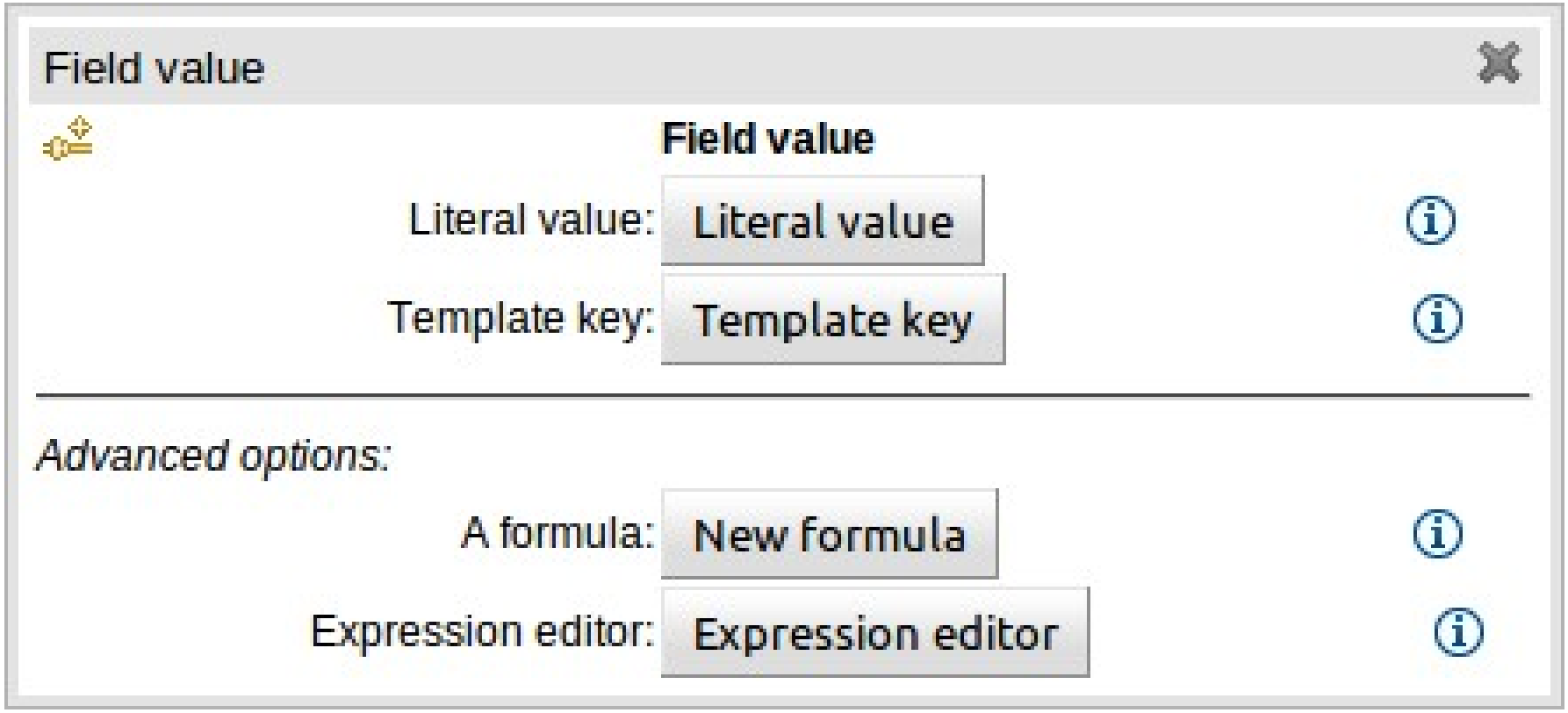
There is an Applicant with:

	age	less than	$max_age
1.	age	greater than or equal to	$min_age
	creditRating	equal to	$cr

THEN

(show options...)

Clés des gabarits



Données du gabarit

Guided Template [t1]

Add row...

$max_age	$min_age	$cr
25	20	AA
		OK
		Sub prime
35	25	AA
		OK
		Sub prime
45	35	AA
		OK
		Sub prime

Éditeur de tables de décision

Drools-WB propose un éditeur pour les tables de décision.

L'éditeur propose les faits et les champs disponibles dans le contexte du projet

2 types de tables peuvent être créées :

- **Entrées étendues** : les définitions de colonnes ne spécifient pas de valeur. Les valeurs sont alors indiquées dans le corps de la table. Elles peuvent cependant être restreintes par un intervalle.
- **Entrées limitées** : les définitions de colonnes spécifient une valeur. Le corps du tableau contient des cases à cocher

Table étendue

Decision table

#	Description	Age	Make	Premium
		Applicant [$a]	Vehicle [$v]	
		age [<]	make [==]	
1		35	BMW	1000
2		35	Audi	1000

Table limitée

Decision table

#	Description	Age < 35	BMW	Audi	Premium 1000
		Applicant [$a]	Vehicle [$v]		
		age [<35]	make [==BMW]	make [==Audi]	
1		☑	☑	☑	☐
2		☐	☑	☑	☐
3		☑	☐	☑	☐
4		☐	☐	☑	☐
5		☑	☑	☐	☐
6		☐	☑	☐	☐
7		☑	☐	☐	☐
8		☐	☐	☐	☐

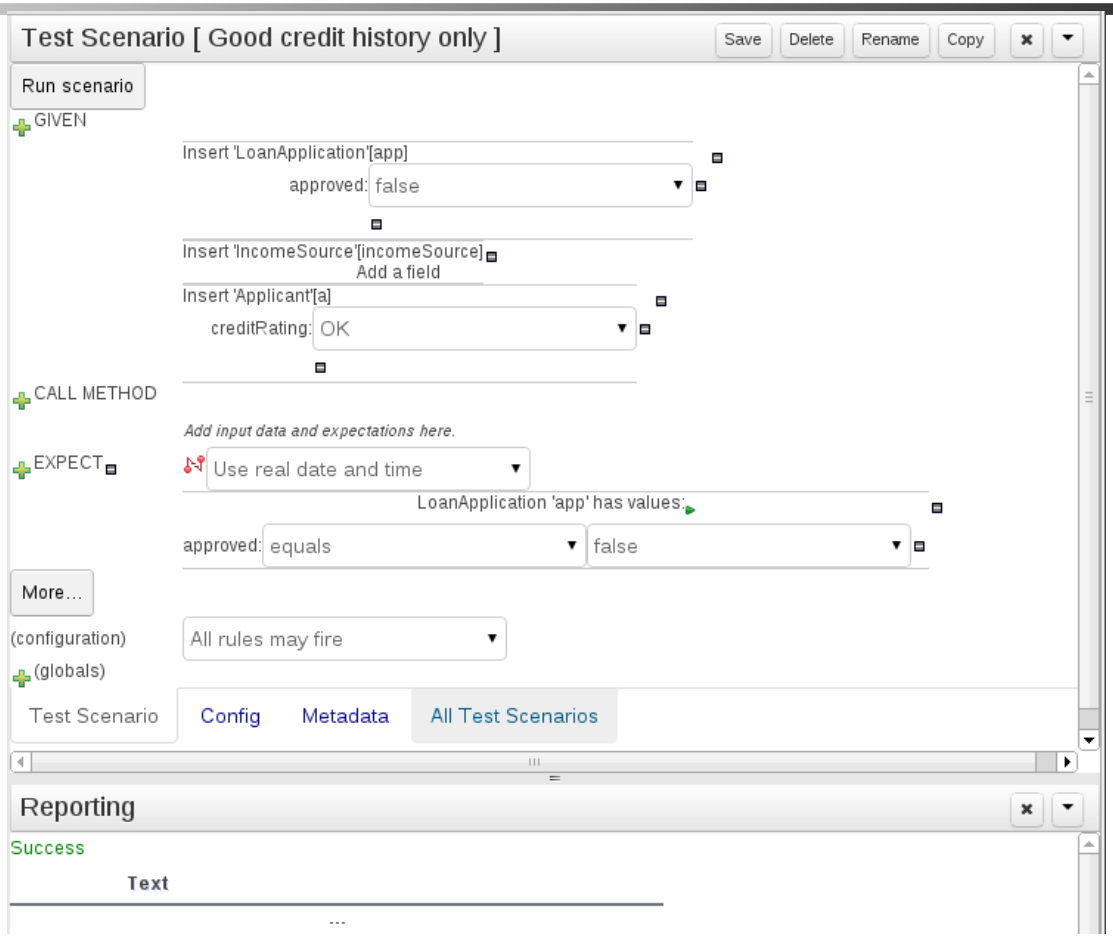
Scénario de test

Les scénarios de Test permettent de valider le fonctionnement des règles et éviter les bugs de régression.

Un scénario de test définit plusieurs sections :

- ***Given*** liste les faits de test
- ***Expected*** liste les changements et actions attendus

Scenario de test



Éditeur DSL

File Edit Source

Attributes Edit

```
[when]When the credit rating is {rating:ENUM:Applicant.creditRating} = applicant:Applicant(creditRating=="{rating}")
[when]When the applicant dates is after {dos:DATE:default} = applicant:Applicant(applicationDate>"{dos}")
[when]When the applicant approval is {bool:BOOLEAN:checked} = applicant:Applicant(approved=={bool})
[when]When the ages is less than {num:1?[0-9]?[0-9]} = applicant:Applicant(age<{num})
[then]Approve the loan = applicant.setApproved(true);
[then]Set applicant name to {name} = applicant.setName("{name}");
```

Liste de valeur

Enum Editor [credit ratings]

Save | Delete | Rename | Copy | Validate

Add enum

	Fact	Field	Context
	Applicant	creditRating	['AA', 'OK', 'Sub prime']
	Person	age	['20','25','30','35']

Accès à la base du Workbench

```
public class MainKieTest {

    public static void main(String[] args) {

        // works even without -SNAPSHOT versions
        String url = "http://localhost:8080/kie-drools/maven2/de/test/Test/1.2.3/Test-1.2.3.jar";

        // make sure you use "LATEST" here!
        ReleaseIdImpl releaseId = new ReleaseIdImpl("de.test", "Test", "LATEST");

        KieServices ks = KieServices.Factory.get();

        ks.getResources().newUrlResource(url);

        KieContainer kieContainer = ks.newKieContainer(releaseId);

        // check every 5 seconds if there is a new version at the URL
        KieScanner kieScanner = ks.newKieScanner(kieContainer);
        kieScanner.start(5000L);
        // alternatively:
        // kieScanner.scanNow();

        Scanner scanner = new Scanner(System.in);
        while (true) {
            runRule(kieContainer);
            System.out.println("Press enter in order to run the test again....");
            scanner.nextLine();
        }
    }

    private static void runRule(KieContainer kieKontainer) {
        StatelessKieSession kSession = kieKontainer.newStatelessKieSession("testSession");
        kSession.setGlobal("out", System.out);
        kSession.execute("testRuleAgain");
    }
}
```

Merci!!!

- MERCI DE VOTRE ATTENTION

Annexes

Algorithmes ReteOO et PHREAK

Algorithme de Rete et ReteOO

Le traitement de comparer les faits aux règles est appelé le **Pattern Matching**, il existe de nombreux algorithmes de pattern matching : *Linear*, *Rete*, *Treat*, *Leaps*.

Drools a dans un premier temps implémenter et optimise l'algorithme de Rete dans un technologie objet. (ReteOO)

Cet algorithme est particulièrement efficace lorsque le jeu de données (les faits) change régulièrement mais dans de faibles proportions car le moteur de règle se souvient des règles ayant déjà matchées

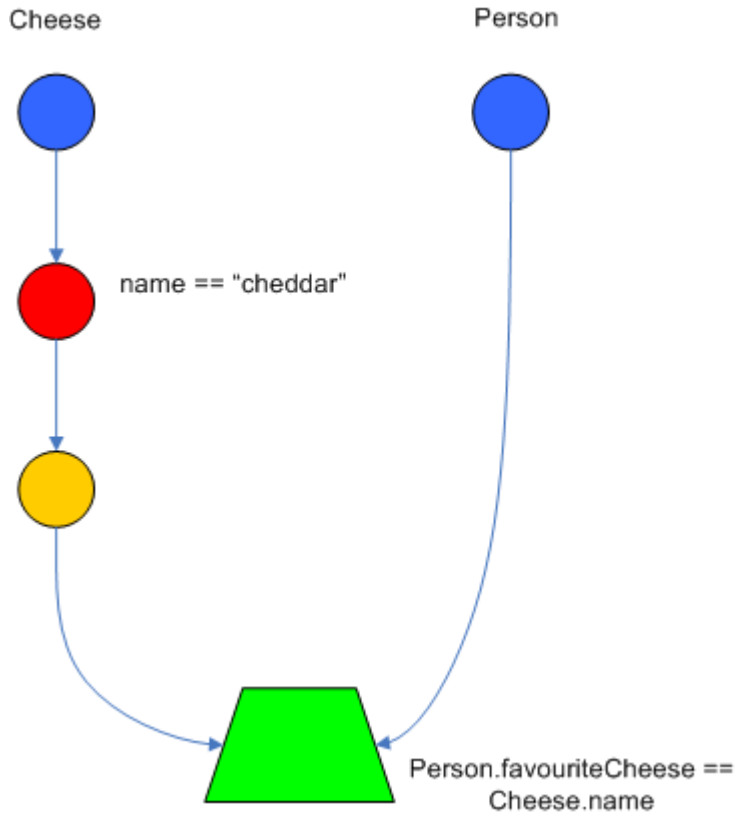
Il est gourmand en mémoire

Principes de Rete

L'algorithme de Rete repose sur 2 étapes :

- La compilation des règles. Elle créée un réseau discriminant constitué de différents nœuds :
  - ***TypeNode*** : Relatif à un type de fait
  - ***Alpha*** : Contrainte sur un fait
  - ***Beta*** : Comparaison entre des tuples d'objets et un objet unique.
  - ***Règle*** : Si une donnée atteint ce type nœud la régle est déclenchée
- Le pattern matching. Il s'exécute à chaque modification de la base de fait. Les faits sont introduits dans le réseau et traverse les nœuds si les conditions sont satisfaites. Certains nœuds (*Beta*) ont une mémoire associée et collecte les faits qui parviennent au nœud.

Exemple


Cheese
Person
name == "cheddar"
Person.favouriteCheese ==
Cheese.name

Chaînage avant / arrière

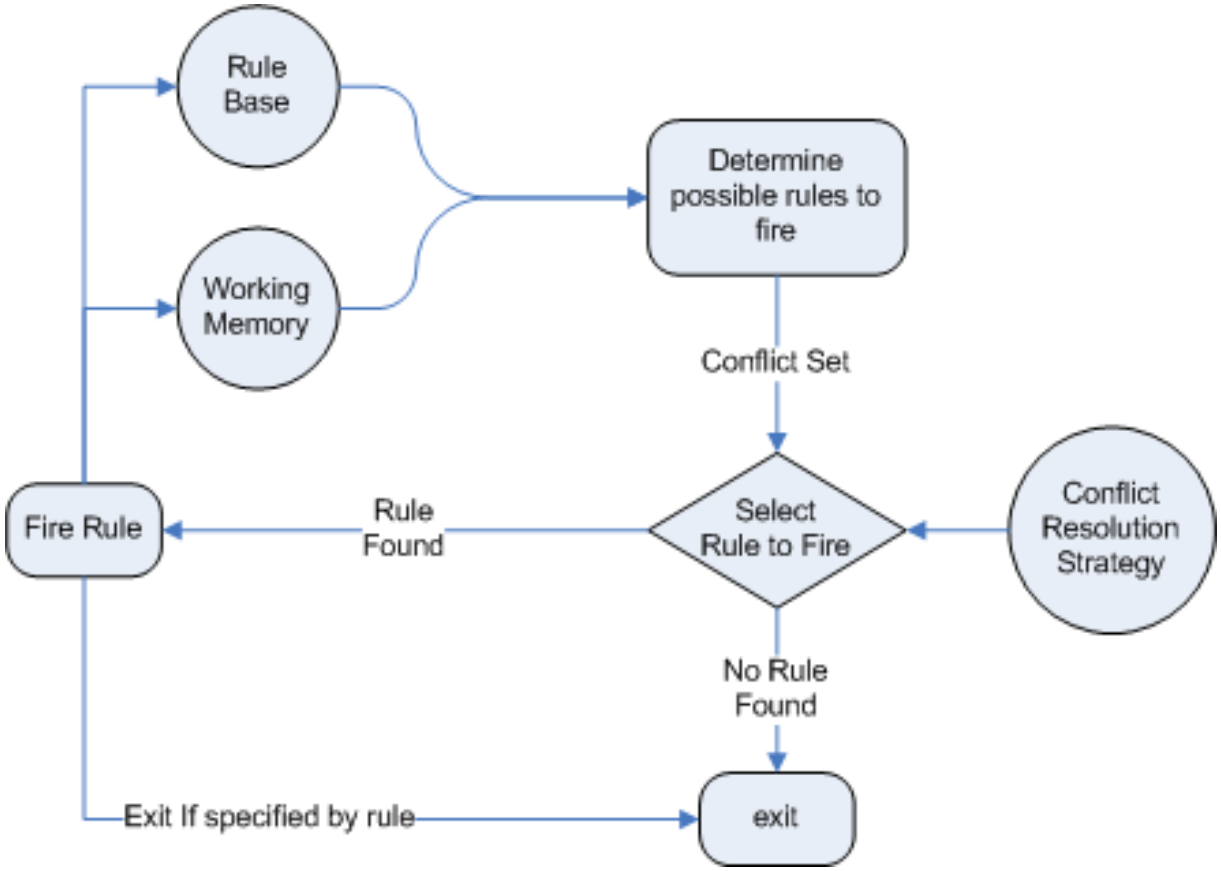
Les moteurs de règles sont à chaînage avant ou arrière :

- Le **chaînage avant** est piloté par les données : à partir d'un fait, des règles s 'appliquent, se propagent et se terminent par une conclusion.
- Le **chaînage arrière** part de la conclusion pour remonter aux causes (Prolog)

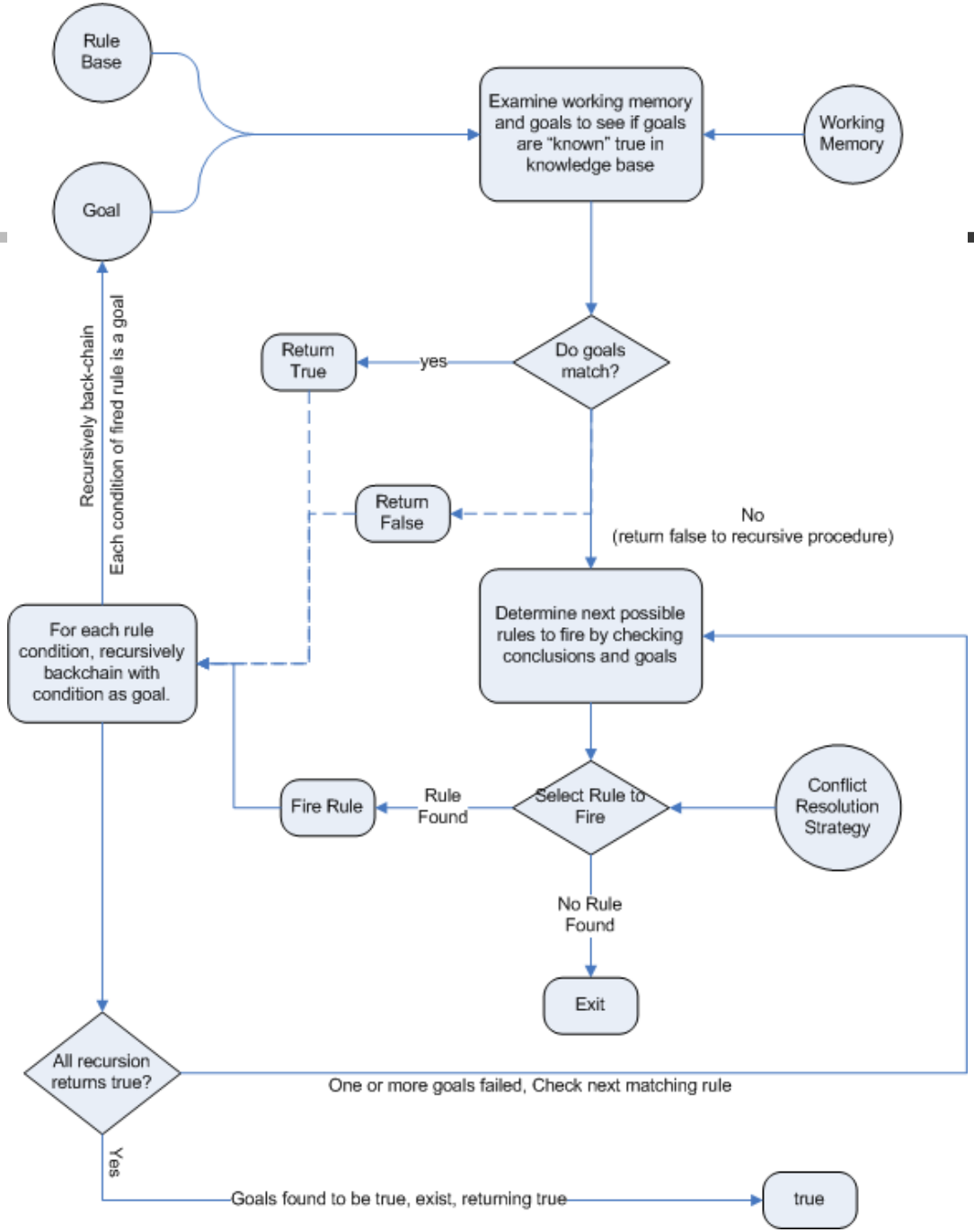
Drools 5 était un moteur à chaînage avant.

Drools 6 est devenu un moteur hybride (avant et arrière)

Chaînage avant



Chaînage arrière



Améliorations de ReteOO

ReteOO est l'algorithme des versions 3, 4 et 5 de Drools.

Il apporte certaines améliorations :

- Performance : Node sharing des nœuds Beta, Indexation Alpha,Beta, Optimisation de la mise à jour de fait, …
- Fonctionnalités : Règles dynamiques

Cependant, le travail de pattern matching est effectué **à chaque modification** de la base des faits (*insert,update,delete*) => Ce qui peut provoquer beaucoup de travail inutile (insertion de fait ne provoquant pas ;'exécution de règles)

Algorithme PHREAK

Drools6 introduit le nouvel algorithme ***PHREAK*** qui améliore *ReteOO*

PHREAK est équivalent à *ReteOO* lorsque le nombre de règles reste modéré mais permet d'éviter des pertes de performances lorsque le nombre de règles grossit

PHREAK permet également des gains de performance lors de l'utilisation de groupes d'agenda et des attributs de priorité (*salience*) des règles

Principes de Phreak

Phreak est lazy, le travail de pattern matching s'effectue seulement lors d'un déclenchement explicite des règles (*fireAllRules*)

Il est à chaînage arrière, chaque règle éligible est évaluée indépendamment des autres

Les techniques de *NodeSharing* sont préservées et des techniques de cache permettent de ne pas re-tester des contraintes n'ayant pas évoluées.

Le *NodeSharing* est implémenté via la segmentation de la mémoire (*Node* → *Segment* → *Rule*), un segment pouvant être partagée par différentes règles

Syntaxe

Élément conditionnel *and*

Les éléments conditionnels peuvent être combinés avec les opérateurs ***and*** (opérateur implicite)

L'opérateur *and* peut-être utilisé en préfixe ou en infixe

Implicite

```
Cheese( cheeseType : type )

Person( favouriteCheese == cheeseType)
```

Préfixe

```
 (and Cheese( cheeseType : type )

Person( favouriteCheese == cheeseType ) )
```

Infixe

```
(Cheese( cheeseType : type ) and

Person( favouriteCheese == cheeseType ) )
```

Élément conditionnel *or*

L'élément conditionnel ***or*** a pour effet de créer plusieurs sous-règles distinctes qui deviennent complètement indépendantes. Il peut s'employer en préfixe ou en infixe.

```
(or Person( sex == "f", age > 60 )
Person( sex == "m", age > 65 )
```

Construction programmatique d’un kModule

Alternative programmatique

- Il est possible de définir les bases de connaissance et les sessions programmatiquement (comme avec Drools 5)
- Il est alors nécessaire de créer un ***KieFileSystem*** et d'y ajouter les ressources

API

KieFileSystem est un système de fichiers mémoire utilisé pour définir programmatiquement les ressources d'un KieModule

KieBuilder est un compilateur pour les ressources contenus dans un *KieModule*

KieRepository : Singleton représentant un dépôt Maven permettant d'accéder aux kieModules via leurs coordonnées

Mise en place

1) Obtenir un *KieFileSystem* à partir du singleton *KieServices*
2) Optionnellement, écrire dans le *KieFileSystem* le *kmodule.xml*
3) Optionnellement, écrire dans le *KieFileSystem* le *pom.xml* (définissant les coordonnées Maven du module)
4) Ajouter une à une les ressources (.drl, .bpmn, …)
5) Faire appel au builder pour compiler les ressources et vérifier les erreurs
6) Obtenir un container à partir du *KieFileSystem*

Création d’un kModule

```
KieServices kieServices = KieServices.Factory.get();
KieModuleModel kieModuleModel =
  kieServices.newKieModuleModel();
KieBaseModel kieBaseModel1 =
  kieModuleModel.newKieBaseModel( "KBase1 ")
.setDefault( true )
.setEqualsBehavior( EqualityBehaviorOption.EQUALITY )
.setEventProcessingMode( EventProcessingOption.STREAM );
KieSessionModel ksessionModel1 =
  kieBaseModel1.newKieSessionModel( "KSession1" )
.setDefault( true )
.setType( KieSessionModel.KieSessionType.STATEFUL )
.setClockType( ClockTypeOption.get("realtime") );
KieFileSystem kfs = kieServices.newKieFileSystem();
kfs.writeKModuleXML(kieModuleModel.toXML());
```

Ajout des ressources

Les ressources peuvent être ajoutées :

- En indiquant leur chemin
- En utilisant la classe *Ressource* de Drools

```
KieFileSystem kfs = ...
kfs.write(
  "src/main/resources/KBase1/ruleSet1.drl",
  stringContainingAValidDRL )
   .write( "src/main/resources/dtable.xls",
kieServices.getResources().newInputStreamResource(
  dtableFileStream ) );
```

Construction

La construction du *kieModule* consiste à fournir le *KieFileSystem* à un ***KieBuilder***.

Si la compilation s'effectue correctement un kModule est stocké dans le *KieRepository* avec ses identifiants Maven :

- Ceux du *pom.xml*
- Ou identifiant par défaut si pas de *pom.xml*

Exemple

```
KieServices kieServices = KieServices.Factory.get();
KieFileSystem kfs = ...
kieServices.newKieBuilder( kfs ).buildAll();
ssertEquals( 0,
  kieBuilder.getResults().getMessages( Message.Level.E
  RROR ).size() );
KieContainer kieContainer =
  kieServices.newKieContainer(kieServices.getRepositor
  y().getDefaultReleasId()) ;
```

Internal *KieHelper*

```
KieHelper kieHelper = new KieHelper();
kieHelper.addResource(ResourceFactory.newClassPathReso
  urce("some/file.drl"), ResourceType.DRL);
//add more resources if needed
Results results = kieHelper.verify();
if (results.hasMessages(Message.Level.WARNING,
  Message.Level.ERROR)){
    //fail
}
KieBase kieBase = kieHelper.build()
```

Marshaller et persistance de session

KieMarshallers

Les ***KieMarshallers*** sont utilisés pour sérialiser/désérialiser les *KieSessions*.

```
ByteArrayOutputStream baos = new
  ByteArrayOutputStream();

Marshaller marshaller =
  KieServices.Factory.get().getMarshallers().newMarshal
  ler( kbase );

marshaller.marshall( baos, ksession );

baos.close();
```

Persistance et transactions

La persistance avec Drools est implémenté via **JPA**.

Il est cependant nécessaire de disposer d'une implémentation de **Java Transaction API** (JTA)

Pour les classes de test et le développement l'implémentation *Bitronix Transaction Manager* est suggérée pour un environnement de production *JBoss Transactions* est recommandé.

Exemple

```
KieServices kieServices = KieServices.Factory.get();
Environment env = kieServices.newEnvironment();
env.set( EnvironmentName.ENTITY_MANAGER_FACTORY,
Persistence.createEntityManagerFactory( "emf-name" ) );
env.set( EnvironmentName.TRANSACTION_MANAGER,
 TransactionManagerServices.getTransactionManager() );
KieSession ksession =
kieServices.getStoreServices().newKieSession( kbase, null, env );
int sessionId = ksession.getId();
UserTransaction ut =
(UserTransaction) new InitialContext().lookup( "java:comp/UserTransaction" );
ut.begin();
ksession.insert( data1 );
ksession.insert( data2 );
ksession.startProcess( "process1" );
ut.commit();
```

Chargement d'une session

Pour récupérer une session stockée en base :

```
KieSession ksession =
kieServices.getStoreServices().loadKieSession(
  sessionId, kbase, null, env );
```

Chargement d'une session

Pour récupérer une session stockée en base :

```
KieSession ksession =
kieServices.getStoreServices().loadKieSession(
  sessionId, kbase, null, env );
```

persistence.xml

```
<persistence-unit name="org.drools.persistence.jpa" transaction-type="JTA">
  <provider>org.hibernate.ejb.HibernatePersistence</provider>
  <jta-data-source>jdbc/BitronixJTADataSource</jta-data-source>
  <class>org.drools.persistence.info.SessionInfo</class>
  <class>org.drools.persistence.info.WorkItemInfo</class>
  <properties>
    <property name="hibernate.dialect"
 value="org.hibernate.dialect.H2Dialect"/>
   <property name="hibernate.max_fetch_depth" value="3"/>
   <property name="hibernate.hbm2ddl.auto" value="update" />
   <property name="hibernate.show_sql" value="true" />
   <property name="hibernate.transaction.manager_lookup_class"
value="org.hibernate.transaction.BTMTransactionManagerLookup" />
  </properties>
</persistence-unit>
```

JTA Datasource avec Bitronix

```
PoolingDataSource ds = new PoolingDataSource();
ds.setUniqueName( "jdbc/BitronixJTADataSource" );
ds.setClassName( "org.h2.jdbcx.JdbcDataSource" );
ds.setMaxPoolSize( 3 );
ds.setAllowLocalTransactions(
ds.getDriverProperties().setProperty("driverClassName",
  "org.h2.Driver");
ds.getDriverProperties().setProperty("url",
  "jdbc:h2:tcp://localhost/JPADroolsFlow");
d1.getDriverProperties().setProperty("user", "sa");
ds.getDriverProperties().setProperty("password", "");
ds1.init();
```

jndi.properties

```
java.naming.factory.initial=bitronix.tm.jndi.BitronixInitialContextFactory
```

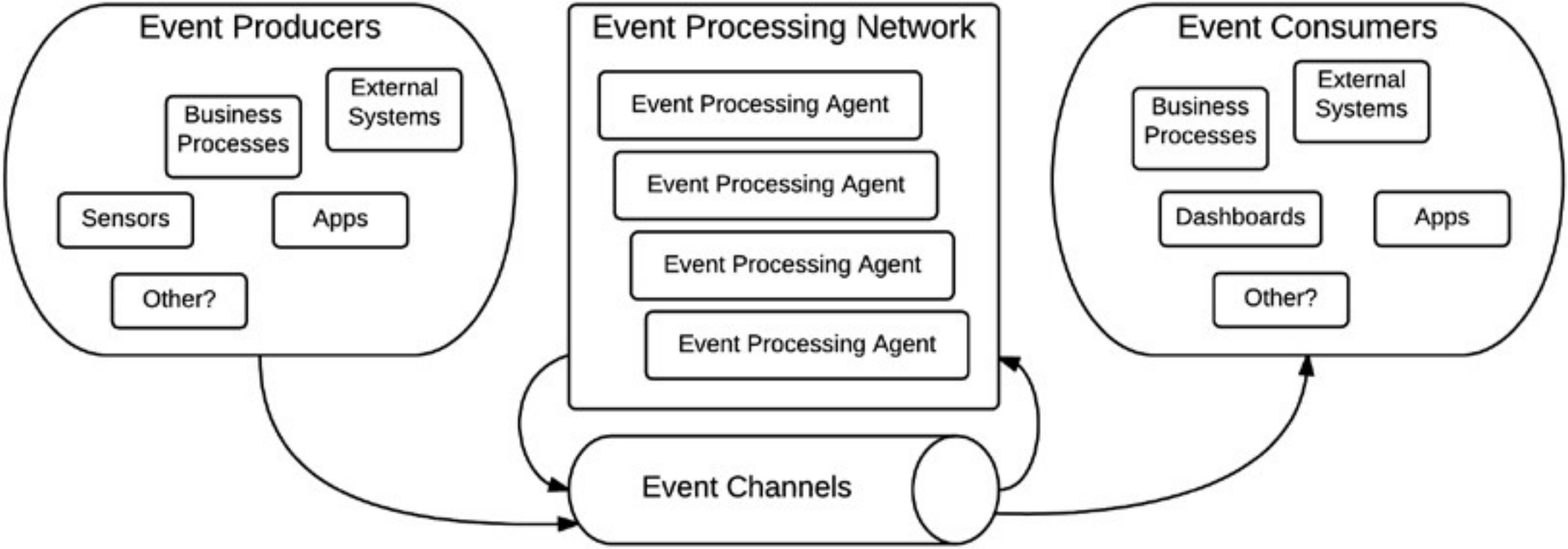
CEP et Event Driven Architecture

Event-driven architecture

The idea of event-driven architecture (EDA) is to classify the components in the following four different categories:

- Event Producer : Creators of events, for example a sensor
- Event Consumer : Final output architecture which point the produced value. For example a dashboard
- Event Channels : Communication protocols between all the other components. For example JMS
- Event Processing Agents: Group the events to detect and process complex events. The Drools rules

Event-driven architecture



Exemple

```
// Insert event from one entry-point to another
rule "Routing transactions from small resellers"
  when
    $t: TransactionEvent() from
      entry-point "small resellers"
  then
    entryPoints["Stream Y"].insert(t);
end
```

Intégration Spring

kie-spring

Utilisation d'un kmodule / Bean configuration

utilisation de balise de namespace kie

Le préfixe a changé de 'drools:' pour 'kie:'

Dépendance :

<groupId>org.kie</groupId>

<artifactId>kie-spring</artifactId>

<version>6.2.0.Final</version>

kie-spring

`<kie:kmodule>`

id requis l'id du bean

ne peut contenir que

`<kie:kbase>`

name : requis

packages

includes

default

scope

eventProcessingMode

equalsBehavior

declarativeAgenda

kie-spring

ne peut contenir que kie:ksession

<kie:ksession>

name : requis

type

default

clockType : REALTIME / PSEUDO

listeners-ref

Déclaration nécessaire au fonctionnement de kie-spring Impératif:

```
<bean id="kiePostProcessor"
        class="org.kie.spring.KModuleBeanFactoryPostProcessor"/>
```

ou pour les annotations

```
<bean id="kiePostProcessor"
        class="org.kie.spring.annotations.KModuleAnnotationPostProcessor"/>
```

kie-spring

exemple

```
<kie:kmodule id="sample-kmodule">
  <kie:kbase name="drl_kiesample3" packages="drl_kiesample3">
    <kie:ksession name="ksession1" type="stateless"/>
    <kie:ksession name="ksession2"/>
  </kie:kbase>
</kie:kmodule>

<bean id="kiePostProcessor"
        class="org.kie.spring.KModuleBeanFactoryPostProcessor"/>
```

kie-spring

<kie:releaseId>

id : bean id, requis

groupId : requis

artifactId : requis

version : requis

exemple

```
<kie:releaseId id="beanId" groupId="org.kie.spring"
        artifactId="named-artifactId" version="1.0.0-SNAPSHOT"/>

<kie:import />
```

releaseId

enableScanner

scannerInterval

kie-spring

Utilisation avec releaseID

```
<kie:import releaseId-ref="namedKieSession"
        enableScanner="true" scannerInterval="1000"/>

<kie:releaseId id="namedKieSession" groupId="org.drools"
        artifactId="named-kiesession" version="6.3.0-SNAPSHOT"/>
```

Le scanner peut être récupéré via

```
KieScanner releaseIdScanner =
  context.getBean("namedKieSession#scanner", KieScanner.class);
releaseIdScanner.scanNow();
```

kie-spring

Listeners

Drools supporte trois types de listeners AgendaListener, WorkingMemoryListener, ProcessEventListener

Ils peuvent être déclaré respectivement via

```
<kie:agendaEventListener/>
<kie:processEventListener/>
<kie:ruleRuntimeEventListener/>
```

DebugAgendaEventListener

DebugRuleRuntimeEventListener

DebugProcessEventListener

Qui sortent les traces sur System.err

kie-spring

ref : référence optionnelle au bean
avec référence ils utilisent l'impémentation fournie

```
<bean id="mock-agenda-listener" class="mocks.MockAgendaEventListener"/>
<bean id="mock-rr-listener" class="mocks.MockRuleRuntimeEventListener"/>
<bean id="mock-process-listener" class="mocks.MockProcessEventListener"/>

<kie:kmodule id="listeners_kmodule">
  <kie:kbase name="drl_kiesample" packages="drl_kiesample">
    <kie:ksession name="ksession2">
      <kie:agendaEventListener ref="mock-agenda-listener"/>
      <kie:processEventListener ref="mock-process-listener"/>
      <kie:ruleRuntimeEventListener ref="mock-rr-listener"/>
    </kie:ksession>
  </kie:kbase>
</kie:kmodule>

<bean id="kiePostProcessor"
          class="org.kie.spring.KModuleBeanFactoryPostProcessor"/>
```

kie-spring

On peut sortir les listener de la session et en faire un groupe disponible pour tous

```
<bean id="mock-agenda-listener" class="mocks.MockAgendaEventListener"/>
<bean id="mock-rr-listener" class="mocks.MockRuleRuntimeEventListener"/>
<bean id="mock-process-listener" class="mocks.MockProcessEventListener"/>

<kie:kmodule id="listeners_module">
  <kie:kbase name="drl_kiesample" packages="drl_kiesample">
    <kie:ksession name="statelessWithGroupedListeners" type="stateless"
          listeners-ref="debugListeners"/>
  </kie:kbase>
</kie:kmodule>

<kie:eventListeners id="debugListeners">
  <kie:agendaEventListener ref="mock-agenda-listener"/>
  <kie:processEventListener ref="mock-process-listener"/>
  <kie:ruleRuntimeEventListener ref="mock-rr-listener"/>
</kie:eventListeners>
```

kie-spring

Logger

se définit dans la session

Console logger

<kie:consoleLogger/>

File logger

<kie:fileLogger/>

ID : indentifiant unique, requis

file : chemin complet ou fichier, requis

threaded

interval

example :

```
<kie:fileLogger id="tfl_logger" file="#{ systemProperties['java.io.tmpdir'] }/log2"
 threaded="true" interval="5"/>
```

Il faut fermer le logger programmatiquement

LoggerAdaptor adaptor = (LoggerAdaptor) context.getBean("fl_logger");

adaptor.close();

kie-spring

<kie:batch>

A placer dans kie:session

contient

- **insert-object**
  - ref = String (optional)
- **set-global**
  - identifier = String (required)
  - reg = String (optional)
- **fire-all-rules**
  - max : n
- **fire-until-halt**
- **start-process**
- identifier = String (required)
- ref = String (optional)
- **signal-event**
  - ref = String (optional)
  - event-type = String (required)
  - process-instance-id =n (optional)

kie-spring

exemple

```
<kie:batch>
  <kie:insert-object ref="person2"/
>
  <kie:set-global
identifier="persons" ref="personsList"/>
  <kie:fire-all-rules max="10"/>
</kie:batch>
```

kie-spring

Persistence

Il faut définir kie:jpa-persistence

puis un transaction manager

puis un entity manager factory

kie-spring

exemple complet:

```
<kie:kstore id="kstore" /> <!-- provides KnowledgeStoreService implementation
 -->

<bean id="myEmf"

 class="org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean">
   <property name="dataSource" ref="ds" />
   <property name="persistenceUnitName"
       value="org.drools.persistence.jpa.local" />
</bean>

<bean id="txManager" class="org.springframework.orm.jpa.JpaTransactionManager">
   <property name="entityManagerFactory" ref="myEmf" />
</bean>
```

kie-spring

```
<kie:kmodule id="persistence_module">
  <kie:kbase name="drl_kiesample" packages="drl_kiesample">
    <kie:ksession name="jpaSingleSessionCommandService">
      <kie:configuration>
         <kie:jpa-persistence>
           <kie:transaction-manager ref="txManager"/>
           <kie:entity-manager-factory ref="myEmf"/>
         </kie:jpa-persistence>
      </kie:configuration>
    </kie:ksession>
  </kie:kbase>
</kie:kmodule>

<bean id="kiePostProcessor"
          class="org.kie.spring.KModuleBeanFactoryPostProcessor"/>
```

kie-spring

Spring Profiles

On peut défnir dans le même fichier des configurations suivant les environnements

en encadrant les tags kmodule par

<beans profile="environnement">

<kie:kmodule

..

</beans>

à spécifier ensuite dans le system.properties ou équivalent

spring.profiles.active="production"

ou par option

-Dspring.profiles.active="development"